QueryAPIGate DOCUMENTATION · REFERENCE

# API reference

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=API

Base URL when running locally: `http://127.0.0.1:5000`. The same information is available interactively at `/docs` and as an OpenAPI document at `/openapi.json`.

If the server has any key configured - `QUERYAPIGATE_API_KEY` or a scoped key created through `/api/v1/api-keys` - send it with every request as `X-API-Key: <key>` (`/health`, `/docs`, `/console` and `/openapi.json` are always public, and `/metrics` unless `QUERYAPIGATE_METRICS_TOKEN` is set; `/openapi.json`'s saved-query section still varies with who's asking). See Authentication and permissions.

| ENDPOINT | METHOD | PURPOSE |
|---|---|---|
| `/execute_sql` | POST | Run ad-hoc SQL |
| `/q/<name>` | GET, POST | Run a saved query as an endpoint |
| `/connections/<name>/schema` | GET | List a connection's tables/views and their columns |
| `/catalog` | GET | The saved queries this caller can reach, and the terms they're offered under |
| `/events` | GET | Live events: every run, as it happens (Server-Sent Events) |
| `/api/v1/...` | GET, POST, PATCH, DELETE | The Management API: saved queries, connections, API keys, roles, collections, history, audit, settings and more (admin only) |
| `/health` | GET | `{"status": "ok", "version": "...", "time_zone": "Asia/Kolkata", "utc_offset": "+05:30"}` - the zone every timestamp the server returns is in (`time_zone` is `null` when the server can't name it) |
| `/metrics` | GET | Prometheus text-format metrics |

## Common query parameters

These apply to every endpoint that returns rows.

| PARAMETER | DEFAULT | DESCRIPTION |
|---|---|---|
| `format` | `json` | `json`, `ndjson`, `csv`, `tsv`, `xml`, `yaml`, `xlsx` or `parquet`. For the POST endpoints it may also be given in the JSON body. |
| `page` | `1` | 1-based page number. |
| `page_size` | `10` | Rows per page, at most `QUERYAPIGATE_MAX_PAGE_SIZE` (default 1000). |
| `timeout` | server limit | Seconds the query may run before it is cancelled with a 504. It can lower the server limit (`QUERYAPIGATE_QUERY_TIMEOUT`, default 30; `0` disables it) but never raise it. For the POST endpoints it may also be given in the JSON body. |

Paging happens *within* the statement's own row window. A query that ends in `LIMIT 3` returns at most 3 rows whatever `page_size` is, and `LIMIT 25` with `page_size=10` gives pages of 10, 10 and 5, then `X-Has-More: false`. Every form is understood: `LIMIT n [OFFSET m]`, `LIMIT m, n`, `LIMIT ALL`, `OFFSET m` alone, and `[OFFSET m ROWS] FETCH FIRST n ROWS ONLY`. Each number may be a bound `:parameter`, whose value must then be a non-negative whole number. A `LIMIT` inside a subquery, a string or a comment is left alone. Responses carry `X-Page`, `X-Page-Size` and `X-Has-More` (`true` when another page exists). A query that returns no rows answers `{"message": "No results returned"}`.

## Execute SQL

`POST /execute_sql?format=json&page=1&page_size=5`

```json
{
    "sql": "SELECT * FROM film WHERE film_id > :min AND title LIKE :name",
    "params": {"min": 10, "name": "C%"},
    "connection_name": "examples"
}
```

- `:name` markers are **bound parameters**: values are sent to the database separately from the SQL. Markers inside string literals, comments and Postgres `::` casts are ignored. Every marker needs a value in `params`.
- Unless the server sets `QUERYAPIGATE_ALLOW_WRITES=1`, only a single `SELECT`, `WITH`, `SHOW`, `DESCRIBE`, `EXPLAIN` or `VALUES` statement is accepted (403 otherwise; 400 for several statements).

## Streaming exports

`page_size` is capped (`QUERYAPIGATE_MAX_PAGE_SIZE`, default 1000) - there is normally no way to pull a full result larger than that in one call. `?stream=true` lifts that cap: the whole result is streamed straight from the database cursor as it comes in, rather than built up in memory first, so an export far larger than fits in memory can still be downloaded. It works on both `POST /execute_sql` and `GET` / `POST /q/<name>`:

```shell
curl -X POST 'http://127.0.0.1:5000/execute_sql?stream=true&format=csv' \
     -H 'Content-Type: application/json' \
     -d '{"sql": "SELECT * FROM film", "connection_name": "examples"}' -o film.csv
```

- Only `format=csv`, `tsv`, `ndjson` or `parquet` are streamable (400 for `json` / `xml` / `yaml` / `xlsx` - those formats all need the whole document structure in memory to write correctly, so paging still applies to them normally). Parquet is written to a temporary file by DuckDB first - its footer comes last - and then sent, so its first byte arrives once the query has finished; memory stays within DuckDB's own, which spills to disk.

### Parquet

`?format=parquet` (paged or streamed) and `queryapigate export --format parquet` work on every database type. The rows are written by DuckDB - the `duckdb` extra, included in the Docker image; without it the request answers `500` `format_unavailable`. Column types come from the values: integers `BIGINT` (`DECIMAL(38,0)` past 64 bits), floats `DOUBLE`, decimals `DECIMAL(38, scale)` - exact, not converted to floats - booleans, `DATE`, `TIMESTAMP` (`TIMESTAMPTZ` with a time zone) and `TIME`; anything else, or a mix, `VARCHAR`. A database that returns dates as text (SQLite) gives text columns, as its driver does. Content type `application/vnd.apache.parquet`.

- `page` / `page_size` are rejected together with `stream=true` (400) - the whole point is that there is no page.
- **Always read-only**, regardless of `QUERYAPIGATE_ALLOW_WRITES` or the calling key's own write permission - a large export has no business mutating data. A write statement gets the usual 403 from the SQL guard.
- The response carries `Content-Disposition: attachment` with a filename, so a browser hitting the URL directly downloads it rather than navigating in-page.
- For a saved query, the run is still appended to `execution_history` once the stream finishes, but `cache_ttl` is ignored - caching would mean building the whole body in memory first, exactly what streaming avoids.
- If the connection or SQL itself is invalid, that surfaces as the usual JSON error before any data is sent. A failure *partway through* an already-started stream cannot change the response's status or body shape any more, though - the client just sees the download end early; check the server log for what actually happened.
- How much this bounds the *server's* memory, not just removing the cap, varies by database - MySQL, PostgreSQL and ClickHouse stream from the server without buffering the whole result client-side first; SQLite, H2, the generic `jdbc` type and DuckDB still bound this project's own memory to one batch at a time, but the underlying engine or driver may materialise more than that internally - see Connection pooling for the per-dialect detail and what it means for how long a pooled connection stays checked out.
- `QUERYAPIGATE_STREAM_MAX_ROWS`, if set, caps how many rows a single export returns - unlike `page_size`, nothing bounds `stream=true` by default, since the whole point is not buffering the result to know its size up front. A malformed value is rejected at startup, the same as `QUERYAPIGATE_RATE_LIMIT`. Past the cap, the export ends early (whatever HTTP headers and rows already went out stand; there's no way to retroactively mark an in-progress `200` as partial) and a `WARNING` is logged naming the connection and the limit hit. `GET /metrics`'s `queryapigate_stream_exports_total` counts a capped export under `status="truncated"`, distinct from `"success"`; a saved query's `execution_history` still records it as `"success"` with the truncated row count - it did succeed, just not to completion.
- **MongoDB** streams too, from 0.37: `?stream=true` on a saved `mongo` query and on `POST /execute_mongo`, with the same limits, grants and history. Documents don't share a schema, so NDJSON writes each one whole, Parquet makes every field a column, and CSV and TSV take their columns from the projection or the first 1,000 documents - a field first seen later is left out and counted in the run's history as `fields_dropped` (MongoDB connections).

## Exports to object storage

> **Stable** since 0.20, under the compatibility promise (what that means). Formats, path placeholders and fields on destinations, exports and runs may still be added. Limit: credentials are given - AWS instance roles aren't used.

A saved query's full result, delivered as a file to a bucket or a folder - Parquet, CSV or NDJSON, written by DuckDB in steady memory (ADR 0004). Two resources:

**A destination** is where exports may write: an `s3://`, `gs://` or `r2://` prefix, or a local folder (only inside `QUERYAPIGATE_EXPORT_DIRS`, and never the server's home), and the credentials to write there - stored like a connection password (masked, `${VAR}`, or encrypted with `QUERYAPIGATE_SECRET_KEY`). Every write goes through a DuckDB connection locked to that prefix, so nothing can be written anywhere else.

```json
{"name": "partner-acme", "url": "s3://acme-exchange/from-us/", "region": "eu-west-1",
 "user": "AKIA...", "password": "${ACME_SECRET_KEY}"}
```

**An export** is a saved query, its parameters, a format, a destination and an object path:

```json
{"name": "acme-daily-orders", "query": "orders_since", "params": {"region": "EU"}, "format": "parquet",
 "destination": "partner-acme", "path": "orders/{date}/orders_{run}.parquet",
 "incremental": {"column": "updated_at", "parameter": "since", "start": "2026-01-01"}}
```

- It runs the query's **published** version. `path` may use `{name}`, `{date}`, `{time}`, `{run}` (the run's id) and the query's parameters (`{region}`, plain values only).
- **Incremental:** each run binds the largest `column` value it last delivered (the **watermark**; `start` the first time) to `:since`, so the query - `WHERE updated_at > :since` - returns only what changed. The watermark moves only after the file is written: a failed run is sent again next time, never lost. `PATCH` with `{"watermark": ...}` (or `null`) rewinds or resets it.
- `skip_empty` (default `true`): a run with no rows writes no file. `timeout`: seconds, within the server's limit.

| METHOD AND PATH | WHAT IT DOES |
|---|---|
| `GET`, `POST /api/v1/destinations` | List; create one (`destination_exists`) |
| `GET`, `PATCH`, `DELETE /api/v1/destinations/{name}` | One destination; change it (the masked secret sent back keeps it); remove it - refused while an export uses it (`destination_in_use`). `If-Match` |
| `POST /api/v1/destinations/{name}/test` | Write one small probe object (`_queryapigate_probe.csv`) under the prefix: url, credentials and permission at once. `POST /api/v1/destinations/test` does it for fields not saved yet |
| `GET`, `POST /api/v1/exports` | List; create one - checked against the destination and the published query (`export_exists`) |
| `GET`, `PATCH`, `DELETE /api/v1/exports/{name}` | One export, with its `watermark` and whether it is `running`. `If-Match` |
| `POST /api/v1/exports/{name}/runs` | Run it now; answers `201` once the object is written: `{"id", "object", "rows", "bytes", "watermark": {"from", "to"}}`. `409 export_running` (with `retry_after`) while another run of it holds the lease |
| `GET /api/v1/exports/{name}/runs` | Its newest runs, from run history |

From the server's shell, with no server running: `queryapigate exports run NAME`, `queryapigate exports list`, and `queryapigate export QUERY --to DESTINATION --out PATH` for a one-off.

**Memory:** a run streams the rows through the server once, into a temporary file, and DuckDB writes from it on one thread - about 150 MB for a local file whatever the size, 180-300 MB to S3 for 2-6 million rows (measured). The step by step: deliver a daily file to a partner's bucket.

Scheduling stays with cron, a Kubernetes CronJob or Airflow. Each run is in run history (`transport: "export"`, with the destination, the object and the watermark), and the `export_failing` alert fires while an export's newest run failed. Owners and admins define destinations and exports (`destinations.write`, `exports.write`) - that decides who receives data; a developer may run one (`exports.run`); every administrator can read them.

## Save a query

`POST /api/v1/queries` creates a saved query with its first version; `POST /api/v1/queries/{name}/versions` adds the next version (see Management API (v1)):

```json
{
    "name": "film_by_id",
    "sql": "SELECT * FROM film WHERE film_id = :id",
    "parameters": {"id": "int"},
    "connection_name": "examples",
    "author": "anantha",
    "description": "Look up a film",
    "tags": ["example"],
    "publish": true
}
```

- `name` may contain letters, digits, spaces, `.`, `_` and `-`. A name already taken is a 409; add a version instead.
- `parameters` declares the query's parameters and their rules. The definitions are checked when you save (400 with an `errors` map if any is invalid), and every parameter declared must be used in `sql`.
- `connection_name` (optional) is the default connection when a run does not name one.
- `cache_ttl` (optional, seconds) caches a response - see Response caching.
- `collection` (optional, on create) files the query under a collection. It belongs to the query, not to a version: change it later with `PATCH /api/v1/queries/{name}`.
- `publish` (default false): a new version is a draft until published - see Drafts and publishing.
- A Mongo query sets `"query_type": "mongo"` and `mongo_collection`, `mongo_filter`, `mongo_projection`, `mongo_sort` instead of `sql`.
- Response: `201` with the query and its versions, and an `ETag`.

## Run a saved query

`GET /q/film_by_id?id=7&format=csv` - query-string arguments other than `format`, `page`, `page_size`, `connection_name` and `version` become parameters.

`POST /q/film_by_id` - the JSON body may contain `params`, `connection_name`, `version` and `format`:

```json
{"params": {"id": 7}, "connection_name": "examples", "version": 1}
```

- The **published** version runs unless `version` is given (see Drafts and publishing).
- `connection_name` from the request wins over the saved default.
- Each run is recorded in that version's `execution_history` (time, connection, status, rows, duration, plus `request_id` / `key_name` - see Observability). Lists show a version's newest 50 runs ( `QUERYAPIGATE_HISTORY_LIMIT` ); how many are kept, and for how long, is configurable - see Run history.

### Drafts and publishing

Every saved query has at most one **published** version: the one `/q/<name>` runs, and the one the catalog, `/openapi.json` , MCP tools, Postman and bundle exports describe. A version newer than the published one is a **draft**, and so is every version of a query with nothing published. A query with nothing published isn't served at all: `/q/<name>` answers 404.

- **Drafts are invisible to callers.** Only the admin key can run one, and only by asking for it by number ( `?version=3` ), to test it before publishing. For any other key, a draft's number answers the same 404 as a version that never existed.
- **Older versions stay runnable** with `?version=` , as before: each was live once.
- **A new version is a draft unless saved with `"publish": true` .** The Management API also publishes, unpublishes and rolls back to any earlier version. `queryapigate collection import` and `examples load` publish what they save.
- **Deleting the published version** falls back to the newest version older than it, never to a draft. With none older, the query is left unpublished.
- `GET /api/v1/queries` shows each query's `published_version` ( `null` when nothing is published).

Stores created before this (schema 3 and older) are upgraded on first start with every query published at its newest version, so nothing a caller sees changes.

`POST /execute_sql_from_file` and `POST /execute_sql_with_parameters_from_file` , which ran a saved query named in the body, were deprecated in 0.14 and removed in 0.15 - use `/q/{name}` .

## Response caching

Opt-in, per saved query: set `cache_ttl` (seconds) when saving it, or turn it on/off or retune it for an existing version in place with `PATCH /api/v1/queries/<name>/versions/<n>` (admin only, `{"cache_ttl": <seconds>}` , `0` or `null` turns it off) - not a new version, so it never bumps `execution_history` or the query's version number, the same way moving a query's collection doesn't. A cached response is only ever served for that exact name, version, connection, resolved parameter values, `format` and page - anything else is a separate entry. It is **never** used for a query whose SQL is a write ( `INSERT` / `UPDATE` / `DELETE` /DDL), regardless of `cache_ttl` : caching such a query would silently skip the write on every call after the first.

A fresh response carries `ETag` , `Cache-Control: private, max-age=<cache_ttl>` and `X-Cache: MISS` - `private` , so the caller's own client may keep it but no shared cache (a CDN, a caching proxy) in front of the server may. A request within the TTL gets the same body with `X-Cache: HIT` ; send back `If-None-Match: <ETag>` to get `304 Not Modified` with no body instead. A cache hit is not appended to `execution_history` - nothing ran against the database. The cache is kept in this one process's memory by default, or shared across restarts/instances when `QUERYAPIGATE_REDIS_URL` is set (see Installation & Setup and the Caching tab in the admin UI) - either way it is shared across every API key that can use the connection, since it stores nothing an authorized caller could not already see by running the query itself.

## Parameter rules

`query_parameters` maps each parameter name to its type, or to an object of rules:

```json
{
    "rating":     {"type": "str", "enum": ["G", "PG", "R"], "default": "PG", "description": "MPAA rating"},
    "max_length": {"type": "int", "min": 1, "max": 600, "default": 120},
    "title":      {"type": "str", "required": false, "min_length": 2, "pattern": "[A-Za-z ]+"},
    "id":         "int"
}
```

| RULE | APPLIES TO | MEANING |
|---|---|---|
| `type` | any | `int`, `float`, `str` or `bool` (aliases `integer`, `number`, `string`, `boolean`). Query-string text is converted; JSON values must already have the right type. Untyped parameters accept any single value. |
| `required` | any | Defaults to `true`, or to `false` when a `default` is given. An optional parameter with no value and no default is bound as `NULL`, so `(:title IS NULL OR title LIKE :title)` works. |
| `default` | any | Used when the request supplies nothing. It must itself satisfy the other rules. Cannot be combined with `"required": true`. |
| `enum` | any | The value must be one of these. |
| `min`, `max` | numbers | Inclusive bounds. |
| `min_length`, `max_length` | text | Length bounds. |
| `pattern` | text | A regular expression the whole value must match (at most 500 characters). |
| `description` | any | Shown in `/docs` and in the MCP tool's input schema - what an agent reads to call it. |
| `example` | any | A sample value, shown in `/openapi.json` and in the MCP tool's input schema. It must itself satisfy the other rules. |
| `embed` | text | Semantic search, experimental: the caller passes text, and the server turns it into a vector (`pgvector`) before binding it. Text only; not combinable with `from_claim`. |
| `from_claim` | any | Take the value from a claim of the caller's sign-in token - e.g. `"sub"` - never from the request. Always required; no `default`. |

Requests that break a rule are rejected with **400** before anything reaches the database, with every problem listed:

```json
{
    "error": "Invalid parameters: rating must be one of: G, PG, R; max_length must be at most 600",
    "errors": {"rating": "must be one of: G, PG, R", "max_length": "must be at most 600"}
}
```

Parameters used in the SQL but not declared still work: they are required and passed through as supplied. Requests rejected this way are not recorded in the query's `execution_history`.

### Text placeholders (legacy)

Saved SQL may also contain `{name}` placeholders, which are substituted **as text** before the query runs. Because the value becomes part of the SQL it must be a number, a boolean, or a string made only of letters, digits, whitespace and `. , : @ % + / -`; anything else is rejected. Prefer bound `:name` parameters.

## List, change and delete saved queries

Through the Management API: `GET /api/v1/queries` lists them (filter by `search`, `collection`, `connection` or `status`), `GET /api/v1/queries/{name}` shows one with every version and its SQL, and `DELETE /api/v1/queries/{name}` (or `.../versions/{n}`, one version - the query goes with its last) removes it. Deleting the published version publishes the newest older one, if any - see Drafts and publishing.

## Management API (v1)

`/api/v1/...` is the versioned interface to QueryAPIGate's own configuration, built one resource at a time (ADR 0001). The Console at `/console` uses it, and so can scripts, Terraform or GitOps tooling. Only administrators may call it, each operation as their role allows. Every request and response is described in full in `/openapi.json`, under the "Management API v1" tag.

**Conventions, the same on every resource:**

- **Errors** are `{"error": "...", "code": "...", "request_id": "..."}`. Branch on `code`, which is stable (for example `query_not_found`, `query_exists`, `version_not_found`, `precondition_failed`, `unknown_field`, `invalid_request`, `unauthorized`, `forbidden`). `error` is for people and its wording can change. `request_id` matches the `X-Request-Id` header and the server log.
- **Optimistic concurrency.** Reading a query returns an `ETag`. Send it back as `If-Match` on a change and the change is refused with `412 precondition_failed` if the query changed in the meantime, through any route.
- **Unknown fields are refused** (`unknown_field`), so a typo never silently does nothing.

## Queries

| METHOD AND PATH | WHAT IT DOES |
|---|---|
| `GET /api/v1/queries` | Every saved query's summary, including `created_at` (its first version), `last_used_at` (its newest stored run) and `cache_ttl`. Filters: `search` (name, description or tag), `collection`, `connection`, `status` (`published`, `unpublished`, `draft`). |
| `POST /api/v1/queries` | Create a query with its first version: `name`, `description`, `sql`, `connection_name`, `parameters`, `tags`, `cache_ttl`, `collection`, `mask` (query-level column masking, SQL only), `mcp` (expose as an MCP tool, default `true`). A draft unless `"publish": true`. 409 `query_exists` if the name is taken. |
| `GET /api/v1/queries/{name}` | The query with every version, each with its `status` (`published`, `draft` or `previous`) and `run_count` (its runs stored in history). |
| `PATCH /api/v1/queries/{name}` | Change its `collection` (`null` removes it). |
| `DELETE /api/v1/queries/{name}` | Delete it and every version. |
| `POST /api/v1/queries/{name}/versions` | Add a version: a draft unless `"publish": true`. The published version keeps serving. |
| `GET /api/v1/queries/{name}/versions/{n}` | One version. |
| `PATCH /api/v1/queries/{name}/versions/{n}` | Change its `cache_ttl` or `mcp` (MCP-tool exposure) in place - not a new version. |
| `DELETE /api/v1/queries/{name}/versions/{n}` | Delete one version; see Drafts and publishing for what happens to the published one. |
| `POST /api/v1/queries/{name}/publish` | `{"version": n}`: publish a draft, or an older version to roll back. |
| `POST /api/v1/queries/{name}/unpublish` | Stop serving it; every version is kept. |
| `GET /api/v1/queries/{name}/versions/{version}/flow` | The tables and joins the version's SQL touches, and the SQL formatted. Best effort: a query that can't be analyzed (Mongo, an unsupported dialect, a parse failure) has an `error` and no tables. |
| `GET /api/v1/queries/{name}/history` | Its runs, newest first. Paged with `limit` and `cursor` (`next_cursor` in the response); filters `version`, `status`, `key`, `since`, `until`. |
| `POST /api/v1/queries/validate` | Check a definition without saving it. Returns every problem found, the parameters the SQL uses and the tables it reads. |

v1 names fields for what they are: `sql` (not `sql_query`), `parameters` (not `query_parameters`), and no `filename`, because `name` is the identity. A parameter rule is the same object as in Parameter rules, with type names `integer`, `number`, `string` and `boolean`. `author` defaults to the calling key's name.

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: <admin key>' -H 'Content-Type:
application/json' \
     -d '{"name": "film_by_id", "description": "One film", "connection_name": "examples",
          "sql": "SELECT * FROM film WHERE film_id = :id", "parameters": {"id": {"type": "integer"}}}'
# a draft: /q/film_by_id answers 404 until it is published
curl -X POST http://127.0.0.1:5000/api/v1/queries/film_by_id/publish -H 'X-API-Key: <admin key>' \
     -H 'Content-Type: application/json' -d '{"version": 1}'
```

## Connections

| METHOD AND PATH | WHAT IT DOES |
|---|---|
| `GET /api/v1/connections` | Every connection's `name`, `db`, `active`, `host`, `port`, `database`, `user`, `example`, timestamps and live `usage` (queries, errors, rows, average latency since the process started). Never credentials. |
| `POST /api/v1/connections` | Create one: `name`, `db` (required), `host`, `port`, `user`, `password`, `database`, `active` (default true), and `options` for driver settings (`sslmode`, `jdbc_url`, ...). 409 `connection_exists` if the name is taken. |
| `GET /api/v1/connections/{name}` | One connection, with its `options`. The password is masked as `********` unless it is a `${ENV_VAR}` reference. |
| `PATCH /api/v1/connections/{name}` | Change some fields; everything not mentioned is kept, driver options included. Send the mask (or leave `password` out) to keep the stored password; `null` removes an optional field. Honours `If-Match`. |
| `DELETE /api/v1/connections/{name}` | Body `{"reason": "..."}`, required (`reason_required`), and kept in the audit log. Every saved query using it stops working. |
| `GET /api/v1/connections/deleted` | Deleted connections, newest first, with who deleted them, when and why. |
| `POST /api/v1/connections/test` | Try to connect with the given fields (an unsaved form), or a saved connection's own by `name`. Nothing is saved. A connection that can't be reached is a 502 `connection_failed` with the driver's message in `detail` (any password in it is redacted). |
| `POST /api/v1/connections/databases` | The databases on that server, for the same two kinds of body. |
| `GET /api/v1/connections/{name}/schema` | Its tables and columns, with their descriptions; `?database=` reads another database on the same server, for the types that have more than one (MySQL, PostgreSQL, ClickHouse, MongoDB). |
| `GET /api/v1/connections/{name}/descriptions` | What its tables and columns mean, and its notes - see Descriptions. |
| `PATCH /api/v1/connections/{name}/descriptions` | Set or remove some: `{"items": [{"table", "column", "description"}]}`. |
| `POST /api/v1/connections/{name}/descriptions/import` | Its database's own table and column comments, as descriptions. |

Every change is audited exactly as the legacy routes audit theirs. A password appears in an audit entry only as `"changed"`, never its value.

## API keys and roles

| METHOD AND PATH | WHAT IT DOES |
|---|---|
| `GET /api/v1/api-keys` | Every key's grants ( `connections` , `queries` , `collections` , `allow_writes` , `allowed_write_ops` , `allowed_tables` , `rate_limit` , `allowed_ips` , `max_concurrent` , `query_timeout` , `max_rows` , `quota` ), `effective_limits` (each against the server's now), `usage_this_period` (with a quota), `active` , `expires_at` and `expired` , `created_at` , `created_from_role` , `last_used_at` and live `usage` ; after a rotation, `previous_expires_at` (the old secret works until then) and `previous_last_used_at` . Never secrets. |
| `POST /api/v1/api-keys` | Create one, from explicit grants or from a `role` (whose grants are copied once; no grant fields then). The response's `secret` is the only time it is ever shown, and is sent with `Cache-Control: no-store` . 409 `key_exists` , 404 `role_not_found` . |
| `GET /api/v1/api-keys/{name}` | One key. |
| `PATCH /api/v1/api-keys/{name}` | Change some grants, `expires_at` or `active` ( `false` revokes it at once); `null` clears an optional field. Honours `If-Match` . |
| `POST /api/v1/api-keys/{name}/rotate` | Reissue its secret: the response carries the new `secret` , shown this once ( `Cache-Control: no-store` ), and the old one keeps working for `grace_minutes` (default 60, at most 10080 - a week; 0 stops it at once). Name, grants, expiry and history stay; `active` , expiry, `allowed_ips` and the rate limit apply to both secrets alike. 409 `key_rotation_pending` while an old secret from the last rotation still works. Audited as `rotate_api_key` . |
| `DELETE /api/v1/api-keys/{name}/previous` | Stop the old secret now, before its grace period ends. 404 `previous_secret_not_found` when none works. Audited as `end_previous_secret` . |
| `DELETE /api/v1/api-keys/{name}` | Revoke and remove it: anything still using it stops working immediately. |
| `GET /api/v1/roles` | Every role's grants, with `keys_created` : the keys created from it. |
| `POST /api/v1/roles` | Create one. 409 `role_exists` . |
| `GET` , `PATCH` , `DELETE` `/api/v1/roles/{name}` | As for keys. A role is a template: changing or deleting it never touches a key already created from it. |

A name is 1-100 letters, digits, spaces, `.` , `_` or `-` ( `invalid_name` ). Every change is audited, never with a secret.

## Administrators

| METHOD AND PATH | WHAT IT DOES |
| --- | --- |
| `GET /api/v1/me` | Who you are: `name`, `role`, `via` (`token`, `break-glass`, `open`), your role's `capabilities`, and `data_access` (whether you may run SQL). Any administrator. |
| `GET /api/v1/administrators` | Every administrator: `name`, `role`, `email`, `active`, `created_at`, `created_by`, `last_seen_at`, `tokens` (how many). Owners only. |
| `POST /api/v1/administrators` | Create one: `{"name": "alice", "role": "developer", "email": "alice@corp.com"}`. 409 `admin_exists`, or `name_taken` for an API key's name. Owners only. |
| `GET`, `PATCH`, `DELETE` `/api/v1/administrators/{name}` | One administrator; change `role`, `email` or `active` (`false` stops all their tokens at once); remove them with their tokens. Honours `If-Match`. 409 `last_owner` when the change would leave no active owner and `QUERYAPIGATE_API_KEY` isn't set. |
| `GET /api/v1/administrators/{name}/tokens` | Their tokens: `id`, `label`, `created_at`, `expires_at`, `expired`, `last_used_at` - never the secret. |
| `POST /api/v1/administrators/{name}/tokens` | Issue one: `{"label": "laptop", "expires_at": "2027-01-01"}` (both optional; no `expires_at`, no expiry). The response's `secret` (`qagadm_...`) is the only time it is shown, sent with `Cache-Control: no-store`. |
| `DELETE` `/api/v1/administrators/{name}/tokens/{id}` | Revoke one; their other tokens keep working. |

Tokens: every administrator may list, issue and revoke **their own**, whatever their role; anyone else's take an owner.

An administrator's `role` is their **server-wide** one, and may be `null` - required on create, so it is a choice: they then have only the roles their workspace memberships give them, listed under `workspaces` (Workspaces). `GET /api/v1/me` adds the `workspace` a call acts in, whether `role` is `server_wide`, and those `workspaces`.

## Workspaces and their members

| METHOD AND PATH | WHAT IT DOES |
|---|---|
| `GET /api/v1/workspaces` | Every workspace - `name`, `description`, `created_at`, `created_by`, `members` (how many) - for a server-wide owner, admin or auditor; the ones you belong to otherwise. |
| `POST /api/v1/workspaces` | Create one: `{"name": "team-a", "description": "Payments"}`. 409 `workspace_exists`. Owners only. |
| `GET`, `PATCH`, `DELETE` `/api/v1/workspaces/{name}` | One workspace (404 for one you may not see, as for one that doesn't exist); change its `description`; remove it - only when it owns nothing (409 `workspace_not_empty`, with what it still has in `contents`), and never `default` (409 `workspace_protected`). Changing and removing: owners only. |
| `GET /api/v1/workspaces/{name}/members` | Who has a role in it: `admin`, `role`, `created_at`, `added_by`. |
| `PUT`, `DELETE` `/api/v1/workspaces/{name}/members/{admin}` | Give an administrator a role in it - `{"role": "developer"}` - or change it; take them out (404 `member_not_found`). A server owner's, or the workspace's own owner's. |

## History and audit

| METHOD AND PATH | WHAT IT DOES |
|---|---|
| `DELETE /api/v1/history?caller=` | Forget a person: delete every run by that caller, its events and its usage counts (`dry_run=true` counts them). Audited as `forget_caller`, by the caller's SHA-256 - pseudonymous: confirmable by someone who knows the name. With several instances, runs another has queued may land a second later: check with `dry_run`. Owners only (`history.forget`); Where data goes. |
| `GET /api/v1/history` | Every saved query's runs, newest first, as `{items, next_cursor}`. Filters: `query`, `version`, `status` (`success` or `error`), `key`, `since` (inclusive) and `until` (exclusive), each a date or a time; `limit` (default 100) and `cursor`. One query's runs alone: `GET /api/v1/queries/{name}/history`. |
| `GET /api/v1/audit` | Administrative changes, newest first: each entry's `timestamp`, `actor`, `action`, `target`, `changes`, and its place in the hash chain (`seq`, `hash`). Filters: `action`, `actor`, `target`, and `q` (matches the time, actor or target). The response also carries `total` (entries stored, before filtering), `actions` (every action in the log) and `retention` (how many entries the log keeps, `QUERYAPIGATE_AUDIT_LOG_LIMIT`). |
| `GET /api/v1/audit/delivery` | Each SIEM output's progress: `delivered_seq`, `behind`, `last_error`, the delivering `instance`. |
| `GET /api/v1/audit/verify` | Walk the audit log's hash chain from its anchor: `{"intact": true, "first_seq", "last_seq", "keyed", ...}`, or `{"intact": false, "break": {"seq", "action", "timestamp", "problem"}}` - `problem` is `changed`, `changed_before`, `missing`, `out_of_order` or `unkeyed_after_keyed`. |

## Collections and the example APIs

| METHOD AND PATH | WHAT IT DOES |
|---|---|
| `GET /api/v1/collections` | Every collection by `name`, with its `queries` and the `keys` and `roles` whose grants reach it; `uncollected` lists the queries in none. File a query under one with `PATCH /api/v1/queries/{name}`. |
| `PATCH /api/v1/collections/{name}` | Rename it: `{"name": "new"}`. Grants follow, and nobody's access narrows part-way. Into an existing collection only with `"merge": true` (409 `collection_exists` otherwise), which also finishes a rename that was interrupted. |
| `GET /api/v1/collections/{name}/postman` | The collection as a Postman Collection v2.1 file. It holds no key. |
| `GET /api/v1/examples` | Whether the example APIs are installed: `loaded`, or `partial` after an interrupted load, and what they are. |
| `POST /api/v1/examples` | Install them (idempotent; completes an interrupted load). The example keys' secrets are in the response, shown this once. 409 `examples_conflict`, changing nothing, if something that isn't an example holds one of their names. |
| `DELETE /api/v1/examples` | Remove exactly what is marked as an example. `keys_still_granted` names other keys granted an example collection, whose grant now reaches nothing. |

## The response cache

| METHOD AND PATH | WHAT IT DOES |
|---|---|
| `GET /api/v1/cache/entries` | What is cached right now (in-process or Redis), soonest to expire first: each entry's `key`, `content_type`, `size_bytes`, `ttl_remaining_s` and `meta` (query `name`, `version`, `connection`, `format`). Never the body. |
| `DELETE /api/v1/cache/entries` | Evict everything: the next call to each query runs it for real. |
| `GET /api/v1/cache/entries/{key}` | The cached body, with its real content type - what a caller receives on a hit. 404 `cache_entry_not_found` once it has expired or been evicted. |
| `DELETE /api/v1/cache/entries/{key}` | Evict one entry early. |

Clearing the cache is housekeeping, not a change, so it is not audited.

## Settings and MCP

| METHOD AND PATH | WHAT IT DOES |
|---|---|
| `GET /api/v1/usage/quotas` | Every caller's `calls` and `rows` for this day and this month - what usage quotas count. |
| `GET /api/v1/settings` | The server's effective configuration, as sections ( `id` , `title` , `description` , `rows` ). Each row has the `env` variable, its displayed `value` , its `source` ( `env` or `default` ) and `env_value` , the raw value for a `.env` export. A secret's `value` only says whether it is configured, and its `env_value` is always null. Read-only: change settings in the environment and restart. |
| `POST /api/v1/settings/rekey` | Re-encrypt every stored password (connections and destinations) under the first key in `QUERYAPIGATE_SECRET_KEY` , so the others can be removed - all in one transaction, or nothing when one can't be decrypted. `{"check": true}` only reports: `keys` , `values` , `by_key` (how many under each listed key, in order), `under_old_keys` , `undecryptable` ; a rekey adds `rekeyed` . Owners only ( `secrets.rotate` ); audited as `rekey` , counts only. See Rotating the key. |
| `GET /api/v1/mcp/status` | `{reachable, port}` : whether something answers on the MCP server's port. A plain TCP connect, run only when you ask. |
| `GET /api/v1/mcp/tools` | What `tools/list` returns for an unrestricted caller: each tool's `name` , `description` , `kind` ( `ad-hoc` or `saved query` ), `params` and `read_only` . Computed in this process, so it works whether or not `queryapigate mcp` is running. |

## Removed routes

Before `/api/v1` existed, QueryAPIGate was managed through unversioned routes. Each was deprecated when its v1 successor shipped, and all were removed together before 1.0, so they are not part of the 1.x contract:

| REMOVED | USE INSTEAD |
|---|---|
| `GET /list_files` , `GET /view_file_content` , `PATCH /save_sql_to_file` , `DELETE /saved_sql/{name}` , `PUT /saved_sql/{name}/collection` , `PUT /saved_sql/{name}/cache_ttl` , `GET /query_flow` | `/api/v1/queries` (the flow is `/api/v1/queries/{name}/versions/{n}/flow` ) |
| `GET /` `PATCH /connections` , `DELETE /connections/{name}` , `POST /connections/test` , `POST /connections/databases` | `/api/v1/connections` |
| `/api_keys` , `/roles` | `/api/v1/api-keys` , `/api/v1/roles` |
| `GET /history` , `GET /audit_log` | `/api/v1/history` , `/api/v1/audit` |
| `GET /settings` , `/settings/mcp_status` , `/settings/mcp_tools` | `/api/v1/settings` , `/api/v1/mcp/status` , `/api/v1/mcp/tools` |
| `/cache/entries` | `/api/v1/cache/entries` |
| `/collections` , `/examples` | `/api/v1/collections` , `/api/v1/examples` |

`GET /connections/{name}/schema` and `GET /connections/{name}/table_ddl` stay: a scoped key may browse the schema of a connection it is granted. A deprecated route carries a `Deprecation` header (RFC 9745) and a `Link: <successor>; rel="successor-version"` header, and `/openapi.json` marks it, for as long as the changelog's deprecation policy says it keeps working.

# Collections

A **collection** is one named group a saved query belongs to - at most one, unlike `tags`, which are free-form and multi-valued. It exists for three things: browsing (the admin UI groups the API Repository list by collection), access (a key can be granted a whole collection instead of a hand-maintained list of names) and moving queries around as a unit (export and import).

**Where it lives.** On the query's own file, as a top-level `"collection"` next to the version numbers (`{"collection": "reporting", "1": {...}}`) - there is no separate registry to fall out of step with the queries. Deleting a query removes its membership; a collection exists exactly while at least one query is in it; moving a query is one atomic file write and **not** a new version, since the SQL did not change. A hand-edited value that is not a valid name is read as *no collection*, so a grant can never reach a query through a spelling the API would have refused.

**Names** are 1-63 characters of lowercase letters, digits, `.`, `_` and `-`, starting with a letter or digit. Anything else - including `Reporting` - is rejected with `400`, never silently case-folded, so two spellings of one collection cannot coexist.

**Moving a query.** `PATCH /api/v1/queries/<name>` (admin only) with `{"collection": "reporting"}`, or `{"collection": null}` to take it out of any. A new query can also be created into one with `collection` in `POST /api/v1/queries`. The audit log records exactly who is affected - a `move_query` entry with the keys that gained and lost access - and the admin UI shows the same before you confirm a move:

```json
{"action": "move_query", "target": "top_rented_films",
 "changes": {"collection": {"from": null, "to": "reporting"}, "keys_gaining_access": ["acme-corp"],
             "keys_losing_access": []}}
```

**Listing.** `GET /api/v1/collections` (admin only) returns every collection with its queries and the keys and roles granted it, plus the queries in no collection. A collection that only a grant still names (its queries have all moved away) is listed with `"queries": []`, so a grant that has gone inert is visible instead of hiding:

```json
{"items": [{"name": "reporting", "queries": ["active_rentals", "top_rented_films"], "keys": ["acme-corp"],
            "roles": ["partner"]}],
 "uncollected": ["loose_query"]}
```

`GET /api/v1/queries` and `GET /catalog` also carry each query's `collection`.

### Granting access to a collection

A key (or role) takes a `collections` list of names:

```json
{"name": "acme-corp", "connections": [], "collections": ["reporting"]}
```

It reaches every query **currently** in those collections, on any connection, and works like `queries`: additive (it only ever adds reach), read-only, and never ad-hoc SQL. Deliberately:

- **No `"*"` wildcard.** "Every query in any collection" would widen itself with each new collection.
- **No write access through a collection.** A write grant stays spelled out per query in `queries`.

- **Names must exist when granted** ( `400` , listing the real ones - a typo would otherwise be a grant that reaches nothing and looks exactly like a working one). Names a key already holds are exempt, so re-saving a key whose collection has since emptied still works.
- **The grant is live**, unlike a role, which is copied once. That is the point - nobody maintains a list - and its cost, since filing a query into a collection changes what every key granted it can run. So every move is written to the audit log with the keys that gained or lost access, and `GET /api/v1/collections` shows who reaches each collection *before* you move anything.

A role's `collections` are copied onto a key at creation like every other role field.

`/openapi.json` , `/catalog` and the run path all decide reachability through one function, so a collection grant is honoured (or not) identically by all three - a test compares them across every kind of grant.

### Renaming a collection

`PATCH /api/v1/collections/<name>` (admin only) with `{"name": "new-name"}` renames it, carrying every key and role grant with it. It is ordered so that no key loses reach at any moment: grants are widened to hold both names, the queries are re-filed, and only then is the old name dropped. If the process dies part-way, access is never narrower than intended and running the same rename again completes it - which is why an already-existing target requires `"merge": true` (409 `collection_exists` otherwise - a half-finished rename looks exactly like one). The response lists the queries, keys and roles changed; the audit log records a `rename_collection` entry.

### Exporting a collection to Postman

`GET /api/v1/collections/<name>/postman` (admin only) downloads the collection as a **Postman Collection v2.1** file, ready for Postman's *Import*; the CLI does the same with `queryapigate collection export <name> --format postman [--base-url https://api.example.com] [--out file]` , and the admin UI has a **Postman** button on each collection's header. Each query becomes a folder-free list of `GET {{baseUrl}}/q/<name>` requests:

- **Parameters** are the query string, named and documented from the query's own rules (type, `enum` , bounds, description). A required parameter is on, with an example that satisfies its rules (its default, the first `enum` value, the lower bound, `true` , …); an optional one is present but switched off, so enabling it is one click. `format` , `page` and `page_size` are always offered, off.
- A `pattern` cannot be turned into an example, so that parameter is left empty and its description says to fill it in. A query with no default connection gets a `connection_name` entry to complete.
- **No credential is ever written.** Authentication is a collection-level `X-API-Key` header from the `{{apiKey}}` variable, which is empty: set it (and check `{{baseUrl}}` , which defaults to the server the file came from, or `http://127.0.0.1:5000` from the CLI) after importing.
- It is a **snapshot of each query's latest version** - export again after queries or their parameters change. It is a file rather than a live link because Postman cannot send an `X-API-Key` header when importing from a URL, and `/openapi.json` shows an anonymous caller none of the saved queries. Export only: there is no import from Postman, since that would mean guessing the SQL - use the bundle to move queries between servers.

### Exporting and importing a collection

```shell
queryapigate collection export reporting --out reporting.json      # or stdout without --out
queryapigate collection import reporting.json --dry-run
queryapigate collection import reporting.json --on-conflict skip
```

A bundle (`"format": "queryapigate-collection"`, `"format_version": 1`) holds the **latest version of each query's definition** - SQL, description, tags, declared parameters, default connection name, `cache_ttl`, any query-level `mask`, and `mcp` exposure - and nothing else: no execution history, no API keys or roles, no connection details (a connection is referenced by name and must exist where you import; missing ones are reported as a warning).

Import is held to exactly the rules saving a query is, and is all-or-nothing up to the point of writing: the whole bundle is validated and checked for conflicts first, so a problem in the tenth query stops the import before the first is written. An unknown field in an entry is an error rather than silently dropped. When a name already exists, `--on-conflict` decides: `fail` (default - import nothing), `skip` (leave the existing query alone), or `new-version` (add the bundle's as the next version). `new-version` never moves a query out of a *different* collection - that would silently change which keys can reach it - so that case is a conflict too. Each query is written atomically and audited as a `save_query` with `"source": "import"`; if the process dies part-way, re-run with `--on-conflict skip` to finish. `--collection NAME` imports into a different collection than the bundle's. Like `queryapigate export`, it runs in-process against `QUERYAPIGATE_HOME` with the same reach as the admin key - no server needs to be running.

## Configuration as code

> **Stable** since 0.30, under the compatibility promise (what that means). Kinds and fields may still be added to `format_version` 1, and commands and options to the CLI; anything else would be a new `format_version`, with this one still read. Limits: the file doesn't hold the server's settings, administrators or their roles (*Not in the file*, below).

One file describes what a server holds - connections, destinations, roles, saved queries, API keys' grants, descriptions and exports - so it can be reviewed as a pull request, kept in git, and promoted from one environment to the next. `queryapigate config export` writes a server's file; `queryapigate config diff` says, field by field, what applying one would change; `queryapigate config apply` makes the server match it.

```shell
queryapigate config export --out queryapigate.yaml        # or stdout; --kinds roles,api_keys for some
of it
queryapigate config diff queryapigate.yaml                # exit 0: no changes, 2: changes, 1: problems
in the file
queryapigate config diff --json queryapigate.yaml         # the same, for a script
queryapigate config apply queryapigate.yaml               # all or nothing; --dry-run, --prune
queryapigate config new-key                               # a secret and its secret_sha256, for a new
key
```

**A file describes one workspace** (Workspaces, experimental): an optional top-level `workspace: team-a` says which; without it, `default` - as every file before workspaces. It is applied there and only there, and it must exist (a file fills a workspace, it doesn't make one). `config export --workspace team-a` writes one, with its `workspace:`; `config diff` and `config apply` go by the file's. Over the API, a file is applied only to the workspace the call is for - `/w/team-a/api/v1/config/apply` - and refused anywhere else. Pruning and what a file manages are per workspace too.

Against a running server, for CI: `POST /api/v1/config/diff` and `POST /api/v1/config/apply` (`?prune=true`, `?dry_run=true`), with the file as JSON or as YAML (`Content-Type: application/yaml`). Owners only - the `config.apply` capability, since a file can change anything; a workspace's own owner applies its file. Or the CLI, with `--server`: it posts the file and shows the answer as it would locally, the key from `QAG_API_KEY` (never an argument, so never in a log), and opens no store of its own:

```shell
QAG_API_KEY=<owner key> queryapigate config diff  --server https://prod.example.com queryapigate.yaml
QAG_API_KEY=<owner key> queryapigate config apply --server https://prod.example.com queryapigate.yaml
```

```yaml
format: queryapigate-config
format_version: 1
connections:
  - name: warehouse
    db: postgres
    host: db1
    port: 5432
    user: reporting_ro
    password: ${WAREHOUSE_PASSWORD}   # a reference, never the password
    database: analytics
roles:
  - {name: partner, connections: [], collections: [finance], rate_limit: 60/minute}
queries:
  - name: monthly_revenue
    collection: finance
    connection_name: warehouse
    description: Revenue per month
    sql: SELECT month, revenue FROM revenue_by_month WHERE month ⩾ :since
    parameters: {since: string}
api_keys:
  - {name: partner-a, role: partner, secret_sha256:
50d858e0985ecc7f60418aaf0cc5ab587f42c2570a884095a9e8ccacd0f6545c}
descriptions:
  warehouse:
    - {table: revenue_by_month, description: "Revenue per calendar month, net of refunds"}
```

Write `${VAR}` references in block style, as above: inside `{...}` on one line, YAML reads the braces as its own.

- **Each object takes the Management API's fields** - what `POST /api/v1/connections`, `/roles`, `/queries` (with `sql` and `parameters`), `/api-keys`, `/destinations` and `/exports` take, and the items of `/connections/{name}/descriptions`, keyed by connection. Each is checked by the API's own rules, and every problem in a file is reported at once, naming the object and the field (`invalid_config`).
- **No secret is ever in it.** A connection's or destination's `password` must be a `${VAR}` reference, expanded where the server runs; a value is refused. An API key carries `secret_sha256`, the hash the store keeps - required for a new key, and a different one is a new secret. An export turns a password the store held as a value into a reference named for it (`${QAG_LEGACY_PASSWORD}`) and lists the variables to set - and, if `QUERYAPIGATE_SECRET_ENV_VARS` is set, for it to allow (`QAG_*`); applying that file reads the password from the server's environment from then on, and the diff says so.
- **A key's `role` stays live**: `role: partner` means the key has `partner`'s grants, so a change to the role in the file changes its keys too (through the API, a role is a template copied once).
- **A section left out is left alone**, and so is what the server has that the file doesn't - the diff lists it. A saved query whose definition changes gets a new version, published; one whose collection alone changes is moved. The examples (`queryapigate examples load`) are exported only with `--examples`.
- **Not in the file:** the server's settings (environment variables), administrators and their tokens, run history, usage counters, the audit log, and a query's earlier versions.

**Applying is all or nothing.** The file is checked and compared with the store, and every change made, in one transaction holding the store's write lock: each through the same function the API uses for it, and audited as the API would audit it - `via: config` - under one `config_applied` entry naming who applied it and the file's SHA-256. A problem at any step (`invalid_config`, naming the object) rolls all of it back, audit entries included. A file that changes nothing writes nothing, so applying the same file twice records nothing the second time.

- A key's new `secret_sha256` is a rotation: the old secret keeps working for 60 minutes, as `rotate` gives it. A new key's secret is the one its hash is of from the start; two keys can't have one secret (`key_secret_taken`).
- **What a file manages:** everything a file has applied, or named, is marked managed (schema 15, `config_managed`). `--prune` **deletes only managed objects** the file no longer has - never what was made in the Console or the API and no file has named - and, for a connection the file lists descriptions for, the descriptions it doesn't list. Deleting is refused where the API refuses it (a destination an export writes to), and changes nothing then.

**At startup, for GitOps.** `QUERYAPIGATE_CONFIG_FILE` names a file the server applies as it starts, before it serves

- once per file: every file applied is recorded by its hash (`config_applied`), so with several instances the first to start applies it, under the store's write lock, and the others find it done; and a change made since in the Console isn't undone at every restart (applying the file by hand does undo it). A file that can't be applied stops startup with its problems, changing nothing - in a rolling update the new pods don't come up, and the ones still serving keep what they have. `QUERYAPIGATE_CONFIG_PRUNE=1` prunes at startup too. The Helm chart's `configAsCode` mounts the file and sets these (DEPLOYMENT.md).

**Locked to the file.** With `QUERYAPIGATE_CONFIG_LOCKED=1`, what a file manages changes only through a file: every `/api/v1` change to one of those objects - an edit, a delete, a new version, a rotation, renaming a collection a managed query is in - answers `409 config_managed`. Running an export, and everything no file manages, works as always. `GET /api/v1/config/managed` lists what's managed and whether it's locked; the Console tags each such object *managed* (or *managed · locked*). Without the lock, a change made in the Console shows as drift in `config diff`.

## Example APIs

Four worked scenarios (reporting, dashboard, export, partner) ship with the package and can be loaded into a home folder and removed again - see Example APIs for the walkthrough. Over HTTP (admin only):

- `GET /api/v1/examples` - `{"loaded": true, "partial": false, "connection": "examples", "queries": [...], "roles": [...], "keys": [...], "collections": [...]}`. `partial` means an interrupted load: some of it is installed.
- `POST /api/v1/examples` - install them. Idempotent; the response lists what was `added`, with the example keys' secrets (`key_secrets`), shown this once. `409 examples_conflict` and nothing changed if a query, role, connection or file that is *not* an example already holds one of their names.
- `DELETE /api/v1/examples` - remove exactly what is marked `example`, and report any keys still granted an example collection (`keys_still_granted`), whose grant now reaches nothing.

The same as `queryapigate examples load|unload|status`, and `QUERYAPIGATE_LOAD_EXAMPLES=yes` loads them at startup (a problem there - a name conflict, a read-only home - is a logged warning, never a startup failure; a malformed value is rejected). Loading records one `load_examples` audit entry, not one per query; removal records `unload_examples`. `GET /api/v1/queries` marks each query with `"example": true/false`.

# Connections

`GET /api/v1/connections` lists connections, and `GET /api/v1/connections/{name}` shows one; stored passwords are shown as `********` (values that are `${ENV_VAR}` references are shown as written). Each entry also carries a live `usage` object - `{"queries": ..., "errors": ..., "rows": ..., "avg_duration_ms": ...}` - aggregated from the same in-process counters `/metrics` renders (see Observability), so the admin UI's Connections tab can show how much a connection has actually been used without a separate Prometheus query. `avg_duration_ms` is `null` until at least one query has run against it since this process started; the numbers reset on restart

- they are a live view of *this process*, not a durable history (see a saved query's own `execution_history` for that).

`POST /api/v1/connections` adds one; `PATCH /api/v1/connections/{name}` changes some of its fields and keeps the rest (`null` removes an optional one). Sending the `********` mask back, or leaving `password` out, keeps the stored password. Driver settings (`sslmode`, `jdbc_url`, …) go in `options`.

```json
{
    "name": "reporting",
    "db": "postgres",
    "host": "db.internal",
    "port": 5432,
    "database": "reports",
    "user": "readonly",
    "password": "${REPORTING_PASSWORD}",
    "active": true
}
```

`DELETE /api/v1/connections/reporting` with `{"reason": "..."}` removes one (the reason is required, and kept in the audit log). `POST /api/v1/connections/test` tries a connection without saving it. See DATABASE_CONNECTION_CONFIGURATION.md for the connection fields.

A `${VAR}` password reference is expanded from the environment at connection time and never written to disk as plaintext. A literal password is encrypted at rest when `QUERYAPIGATE_SECRET_KEY` is set (see Encryption at rest below); without that variable it is stored as given, in `queryapigate.db` on disk, not just masked in API responses. The server logs a startup warning naming any connection whose password is still a literal string with no protection at all, so a deployment that hasn't adopted either convention finds out - nothing blocks it, this is a nudge, not an enforcement.

### Encryption at rest for connection passwords

Set `QUERYAPIGATE_SECRET_KEY` to a Fernet key (`python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`) and every literal connection password - existing ones immediately at startup, new ones the moment they're saved - is encrypted before it touches disk, decrypted only in memory at the instant a connection is actually opened. Needs the `cryptography` package (`pip install "queryapigate[encryption]"`, included in `[all]`); a clear error at startup names the missing package if `QUERYAPIGATE_SECRET_KEY` is set without it. A `${VAR}` reference is untouched either way - it was never a secret stored in the file to begin with.

An encrypted password is masked the same as a literal one in `GET /api/v1/connections/{name}` and the audit log (`********`) - the stored ciphertext itself is never returned to a client. Losing or rotating `QUERYAPIGATE_SECRET_KEY` fails clearly rather than quietly: a connection whose password can't be decrypted returns a `500` naming the problem, and the server logs a startup warning if encrypted passwords exist on disk but no key is configured to read them - the same "fail closed, say why" precedent `expires_at` and `rate_limit` already follow for a key's own malformed data.

There is no way to recover an encrypted password without the key that encrypted it; keep `QUERYAPIGATE_SECRET_KEY` itself somewhere safe, outside `QUERYAPIGATE_HOME` and outside version control, the same way you would any other credential.

### Rotating the key

`QUERYAPIGATE_SECRET_KEY` takes several keys, comma-separated (ADR 0009): the first encrypts, every one decrypts. Restart with `<new-key>,<old-key>`, run `queryapigate rekey` (or `POST /api/v1/settings/rekey`) to re-encrypt every stored password - connections' and destinations' - under the new key, then restart with `<new-key>` alone. `queryapigate rekey --check` only reports how many are under each key. Until the old keys are gone, startup logs a warning and the `encryption_key_rotation_incomplete` alert says what's left. Each listed key is validated at startup, by its place in the list (*Key 2 of 2 in QUERYAPIGATE_SECRET_KEY must be a valid Fernet key*). Step by step, with several instances: How to encrypt passwords at rest.

`GET /connections/reporting/schema` lists its tables and views for self-service query writing:

```json
{
    "tables": [
        {"name": "orders", "type": "table", "columns": [
            {"name": "id", "type": "integer", "nullable": false, "position": 1},
            {"name": "customer_id", "type": "integer", "nullable": false, "position": 2}
        ]}
    ],
    "truncated": false
}
```

`truncated` is `true` only if the connection has more than 5000 columns across all its tables and views combined, in which case the list was cut off.

## Secrets from files and secrets managers

> **Stable** since 0.34 for `${VAR}`, `${file:...}`, `${vault:...}` and `vault-database` credentials, under the compatibility promise (what that means), tested on every push against Vault 1.21 (ADR 0021). Providers, credential sources, Vault sign-in methods and settings may still be added; the reference forms, the `credentials` shape and the allow-lists stay as they are. AWS - `${aws-sm:...}` and `aws-rds-iam` - is still experimental: see Secrets and credentials from AWS. Limits: a resolved value is used for up to `QUERYAPIGATE_SECRET_CACHE_SECONDS` after it's rotated or revoked; what an allowed reference names, an administrator can still send to a host of theirs (bind Vault roles to their hosts); whoever can write inside an allowed folder decides what a `${file:...}` reads; over an `http://` `QUERYAPIGATE_VAULT_ADDR` the server's Vault token crosses the network unencrypted; Vault leases are revoked when a process stops, not when it crashes.

A connection's or destination's field can name where its secret lives instead of holding it - resolved when the connection is opened, kept in memory a while, and never written anywhere. Responses, diffs and configuration files show the reference as written.

| REFERENCE | RESOLVED FROM |
|---|---|
| `${VAR}` | The server's environment - only the variables `QUERYAPIGATE_SECRET_ENV_VARS` lists, when it's set. |
| `${file:/run/secrets/warehouse}` | A file - what the Secrets Store CSI driver, Docker secrets and Vault Agent write. Read again as soon as it changes, so a password rotated in place is used by the next connection; one trailing newline is dropped; at most 64 KiB. |
| `${vault:secret/data/warehouse#password}` | A field of a secret in Vault's KV engine, version 1 or 2: the path as Vault's API names it ( `secret/data/...` for version 2), `#` , the field. |
| `${aws-sm:prod/warehouse#password}` | An AWS Secrets Manager secret, by name or ARN - with `#field` , that field of its JSON; without, the whole string. Experimental: below. |

A reference can stand alone ( `"password": "${file:/run/secrets/warehouse}"` ) or sit inside a value ( `"jdbc_url": "jdbc:...;password=${file:/run/secrets/warehouse}"` ).

**Nothing is reachable until you say so.** A provider reads whatever the server can, which is more than any one connection should: an administrator could point a connection at a host of theirs and have the server send them a secret meant for something else. So `${file:...}` reads only inside `QUERYAPIGATE_SECRET_FILE_DIRS` (comma-separated absolute folders, symbolic links followed) - unset, it's refused - and never the server's home, where its store is. A reference outside is refused when it's saved ( `invalid_body` ), and again when it's used, in case the list shrank ( `connection_misconfigured` ). The server's own `QUERYAPIGATE_*` and `OTEL_*` settings are never reachable, by any reference, and nor are the credentials it may itself run with - `AWS_*` , `AZURE_*` , `ARM_*` , `GOOGLE_*` , `GCP_*` , `VAULT_*` and `KUBERNETES_*` - which reach far more than one connection's secret. Nor, through `${file:...}` , is a file one of them names ( `QUERYAPIGATE_SECRET_KEY_FILE` , `QUERYAPIGATE_VAULT_TOKEN_FILE` , `AWS_WEB_IDENTITY_TOKEN_FILE` …), the Kubernetes and EKS service account tokens, or anything under `/proc` - whatever `QUERYAPIGATE_SECRET_FILE_DIRS` allows. `${VAR}` reads any other variable until `QUERYAPIGATE_SECRET_ENV_VARS` lists the ones it may: names, or prefixes ending in `*` ( `WAREHOUSE_PASSWORD,QAG_*` ). Set it: while references are in use without it, the server says so at startup.

**It fails closed.** A resolved value is kept for at most `QUERYAPIGATE_SECRET_CACHE_SECONDS` (default 300) and fetched again a little before that. If its provider fails then, the value it gave before is used until its time is up - with a warning in the log - and never after: the connection fails with `503 secret_unavailable` . Why - the reference and the provider's error, never the secret - is in the log and in the error's `detail` , which a scoped key calling a saved query doesn't get, as with a database's own error text: it names where the server's secrets live. A provider that failed isn't asked again for 10 seconds, and while one request waits on it the others get what's held - so an outage doesn't make every call wait on a timeout.

**Short-lived database credentials.** Instead of a user and password, a connection can say where they come from:

```json
{"name": "warehouse", "db": "postgres", "host": "warehouse.internal", "database": "analytics",
 "credentials": {"source": "vault-database", "role": "reporting-ro"}}
```

The server asks Vault's database secrets engine for a username and password when a connection is first opened, uses the pair for every connection opened during its lease, and asks again at 80% of it. Each pooled connection is retired before its lease ends - a tenth of the lease, between 5 seconds and 5 minutes, before - so a query never runs on a connection whose user Vault is about to drop. Leases the process took are revoked when it exits cleanly. Any type with a user and password (not SQLite, DuckDB or H2); with `credentials` , leave out `user` and `password` .

**Vault.** `QUERYAPIGATE_VAULT_ADDR` (with `QUERYAPIGATE_VAULT_CACERT` for a private CA, and `QUERYAPIGATE_VAULT_NAMESPACE`), and the way this server signs in, `QUERYAPIGATE_VAULT_AUTH`:

| | NEEDS |
|---|---|
| `token` (default) | `QUERYAPIGATE_VAULT_TOKEN`, or `QUERYAPIGATE_VAULT_TOKEN_FILE` - read again at each sign-in, so the token Vault Agent keeps current there is the one used |
| `kubernetes` | `QUERYAPIGATE_VAULT_K8S_ROLE`; the pod's service account token, from `QUERYAPIGATE_VAULT_K8S_TOKEN_FILE` (the usual path by default) |
| `approle` | `QUERYAPIGATE_VAULT_ROLE_ID` and `QUERYAPIGATE_VAULT_SECRET_ID` (or `_SECRET_ID_FILE`) |

`QUERYAPIGATE_VAULT_AUTH_MOUNT` if the method isn't mounted at its own name, `QUERYAPIGATE_VAULT_DATABASE_MOUNT` if the database engine isn't at `database`. What Vault lets this server's token read is more than any connection should: `${vault:...}` reads only under `QUERYAPIGATE_VAULT_PATHS` (comma-separated prefixes, `secret/data/reporting/`), and `credentials` asks only for `QUERYAPIGATE_VAULT_DATABASE_ROLES` - unset, neither is allowed. **Bind each role to its database's host** - `reporting-ro@warehouse.internal` or `reporting-ro@warehouse.internal:5432` - and its credentials go to no other: a connection naming it with another host is refused when saved, tested or used. Unbound, a role's credentials go to whatever host a connection names. Tested in CI against Vault 1.21 and PostgreSQL; walkthrough: Use short-lived database credentials from Vault.

**When a provider keeps failing**, the `secret_provider_failing` alert (critical) says so after five minutes, and Settings shows how each provider this instance has used last answered.

**The server's own secrets, from files.** Each setting that is a secret - `QUERYAPIGATE_API_KEY`, `_SECRET_KEY`, `_DATABASE_URL`, `_REDIS_URL`, `_AUDIT_KEY`, `_MASK_KEY`, `_JWT_SECRET`, `_SSO_CLIENT_SECRET` - can be read from a file instead, with `_FILE` added: `QUERYAPIGATE_SECRET_KEY_FILE=/run/secrets/fernet`. Read at startup, as the variable would be; setting both is refused. Settings says which came from a file.

### Secrets and credentials from AWS

> **Experimental** - may change in any minor release, always noted in the changelog (what that means): tested in CI against a Secrets Manager stand-in, not AWS, and RDS IAM authentication against neither, so the reference form and settings may still change as they meet real AWS (ADR 0021).

Through boto3 (`pip install "queryapigate[aws]"`; in the image already), with the standard credential chain - IRSA on EKS (the chart's `serviceAccount.annotations`), an instance or task role, the usual `AWS_*` variables - and region (`QUERYAPIGATE_AWS_REGION`, else boto3's). `${aws-sm:...}` reads only secrets whose names begin with one of `QUERYAPIGATE_AWS_SECRET_PREFIXES`, and `aws-rds-iam` makes tokens only for `QUERYAPIGATE_AWS_RDS_HOSTS` (names, or suffixes starting with a dot: `.abc123.eu-west-1.rds.amazonaws.com`) - unset, neither is allowed.

For RDS and Aurora, `"credentials": {"source": "aws-rds-iam"}` makes an IAM authentication token the password of the connection's own `user` - PostgreSQL or MySQL, good for 15 minutes, signed by the server from its own AWS credentials (nothing is sent to AWS to make it). A token is a password, so only over TLS: a PostgreSQL connection needs `sslmode` `verify-full` (with `sslrootcert`, the RDS CA bundle) or `require`, a MySQL one `ssl_ca`. Leave out the `password`. Tested in CI to the token's signature; whether RDS accepts it can only be tested against RDS.

## Descriptions

> **Stable** since 0.30, under the compatibility promise (what that means). Fields - of a description, and where descriptions are served - may still be added; none is removed or renamed except through the deprecation process. Limits: comments are imported from PostgreSQL, DuckDB, MySQL and ClickHouse only; a table's or column's description is up to 2,000 characters and a connection's notes up to 20,000; and another database browsed with `?database=` is shown without them.

An AI agent - or a new colleague - sees `orders.amt` and `cust_tier`; it writes the right SQL only if it's told "net revenue in EUR, refunds excluded". A connection keeps a description for each table and column, and its own **notes** (which joins to use, how a metric is worked out):

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/connections/shop/descriptions -H "X-API-Key: $TOKEN" \
  -H 'Content-Type: application/json' -d '{"items": [
    {"table": "orders", "column": null, "description": "One row per order"},
    {"table": "orders", "column": "amt", "description": "Net revenue in EUR, refunds excluded"},
    {"table": null, "description": "Join orders to customer on customer_id. Revenue is SUM(amt) over
paid orders."}
  ]}'
```

- `table` null is the connection's notes (up to 20,000 characters); `column` null a table's own description; a table's or column's is up to 2,000. A null or empty `description` removes one. Names match case-insensitively.
- **Import what's already there:** `POST /api/v1/connections/{name}/descriptions/import` reads the database's own comments - `COMMENT ON` in PostgreSQL and DuckDB, table and column comments in MySQL and ClickHouse - as descriptions with `"source": "database"`. One written by a person (`"source": "written"`) is kept unless the body says `{"overwrite": true}`. It answers how many it `found`, `imported` and `kept`.
- **Served with the schema:** `description` on each table and column of `GET /connections/{name}/schema` (and `/api/v1`'s), and `notes`; MCP's `list_tables`; and the data dictionary resource. Only for the tables a caller can reach (`allowed_tables`) - and a caller restricted to some tables gets no notes, which may name any table - and a column the caller gets masked says so: `"masked": "hash"` (or `mask`, `null`, `drop`, or `refused` for a view column a rule may mean) - so an agent doesn't plan around a value it will never see. Another database browsed with `?database=` is shown without them.
- **A saved query's own result columns** are described in its definition: `"columns": {"amt": "Net revenue in EUR"}` on `POST /api/v1/queries` (and its versions), carried in its MCP tool's output schema.
- Who may: reading with `connections.read`, changing and importing with `connections.write` (owners and admins). Every change is in the audit log - `update_descriptions` or `import_descriptions`, with what each one said before and after.

In the Console: the schema browser's ✎ beside a table describes it and its columns; ✎ **Describe connection** holds the notes and **Import comments**.

## Administrators and admin roles

The people and pipelines that manage QueryAPIGate are **administrators** (ADR 0003), each with a role, signing in with their own **admin tokens** - `qagadm_...`, sent as `X-API-Key` like any key, stored only as hashes, each with an optional expiry. An administrator may hold several (a laptop, a CI job) and revoke each alone; deactivating the administrator stops them all.

| ROLE | MAY |
|---|---|
| `owner` | everything, including administrators and re-encrypting stored passwords under a new key |
| `admin` | everything but administrators and re-encrypting passwords: connections, API keys and roles, saved queries, cache, examples, settings |
| `developer` | saved queries and collections; read connections, history, the audit log and alerts; run SQL - but no API keys, no connection changes, and no moves that give a key or role new reach |
| `auditor` | read only: queries, connections, API keys and roles, the audit log, history, alerts, instances, settings. No SQL at all |

Owners, admins and developers run SQL and saved queries on every connection, as `QUERYAPIGATE_API_KEY` does. Each `/api/v1` operation needs one capability (`queries.write`, `access.read`, …; `GET /api/v1/me` lists yours); a role without it gets `403 role_forbidden`, naming the `capability`. Any other caller - a scoped key, a signed-in app user - gets `403 admin_only`.

**The first owner** is created on the server, with no running server or key needed:

```shell
queryapigate admins create alice --role owner --email alice@corp.com --label laptop
# Created administrator alice (owner).
# Admin token tok_3f2a9c1b04de for alice (expires 2027-01-03) - store it now, it cannot be shown again:
#   qagadm_...
```

`--expires YYYY-MM-DD|never` (default: 90 days). `queryapigate admins token NAME` issues another (for one who lost theirs); `queryapigate admins list` lists them. Or create administrators in the Console, or through `/api/v1/administrators`, with the shared key.

**`QUERYAPIGATE_API_KEY` is the break-glass key.** It keeps working, as an owner, so nothing breaks on upgrade. Once an active owner exists, each use is logged as a warning and recorded in the audit log (`break_glass_used`, at most every 10 minutes per process), which raises the `break_glass_used` alert for 24 hours. When everyone signs in with their own token, remove it from the environment: **authentication stays required while any administrator exists**, and the last active owner can't be demoted, deactivated or removed while the shared key isn't set (`409 last_owner`).

Administrator names and API key names never overlap (`409 name_taken`), so a name in the logs, `/metrics` and run history means one caller; `admin` and `cli` are reserved.

### Console sign-in through SSO

> **Stable** since 0.30, under the compatibility promise (what that means). Settings, the claims it reads, and fields of `/auth/session` may still be added. Limits: one identity provider at a time, through OpenID Connect (no SAML); and a role comes from the provider's groups at sign-in, so a change there reaches an open session when it ends - `QUERYAPIGATE_SSO_SESSION_HOURS` - unless the administrator is switched off here, which ends it at once.

Administrators sign in to the Console through the organisation's identity provider (OpenID Connect) - its MFA and policies apply, and their role comes from their groups:

| VARIABLE | EFFECT |
| --- | --- |
| `QUERYAPIGATE_SSO_ISSUER` | The provider's issuer URL; its endpoints and keys come from its published metadata |
| `QUERYAPIGATE_SSO_CLIENT_ID`, `QUERYAPIGATE_SSO_CLIENT_SECRET` | This server, registered at the provider as a confidential client |
| `QUERYAPIGATE_SSO_REDIRECT_URL` | The callback as the provider calls it - `https://data.example.com/auth/sso/callback` |
| `QUERYAPIGATE_SSO_ROLES` | Groups to roles: `owner=qag-owners,admin=qag-admins,developer=qag-devs,auditor=qag-audit` |
| `QUERYAPIGATE_SSO_GROUPS_CLAIM` | The ID token claim with the person's groups (default `groups`) |
| `QUERYAPIGATE_SSO_SESSION_HOURS` | How long a session lasts (default 8, at most 168) |
| `QUERYAPIGATE_SSO_DEFAULT_ROLE`, `QUERYAPIGATE_SSO_ALLOWED_DOMAINS` | For providers whose ID tokens carry no groups: a role (never owner) for a verified email at these domains - only together |

- **The flow** is the authorization code flow with PKCE, run by the server: `GET /auth/sso/login` sends the browser to the provider; `GET /auth/sso/callback` takes the sign-in back - once, and only in the browser that started it
  - exchanges the code with the client secret, and checks the ID token: signature, issuer, audience (the client ID), nonce, expiry. The browser never holds a token.
- **Who**: the person is bound to their administrator by the provider's `sub`, never by email; the first sign-in creates the administrator (`via: sso`). **The role is the groups'**, at every sign-in - the most capable that maps; none, and the sign-in is refused (`sso_no_role`).
- **The session** is an `HttpOnly`, `Secure`, `SameSite=Strict` cookie, and a request is authenticated by it only together with the session's CSRF token in `X-QAG-CSRF` - which the Console reads from `GET /auth/session`. Sent with an `X-API-Key` or `Authorization` header, the header is judged instead. `POST /auth/sso/logout` ends it.
- **Administrators who had tokens before** keep their name and history: `PATCH /api/v1/administrators/alice` with `{"sso": "pending"}` binds her at her next SSO sign-in, by a verified email matching hers - once. From then on her role is her groups' (changing it here is `409 sso_managed`); `{"sso": null}` unlinks her. Switching an administrator off, or removing one, ends their sessions at once.
- **`QUERYAPIGATE_SSO_ONLY=yes`**: an administrator who signs in through SSO has no personal tokens - theirs stop working and none can be issued (`sso_only`), so the provider's MFA and policies can't be routed around. Automation accounts, not linked to anyone, keep theirs.
- **A refused sign-in** lands on the Console with `sso_error`: `sso_no_role` (no group with a role), `sso_state` (expired, used, or started in another browser), `sso_id_token` (the provider's answer didn't verify), `sso_inactive`, `sso_denied` (the provider ended it), `sso_unreachable`, `sso_ambiguous`.
- Every sign-in, refusal and sign-out is in the audit log, and every action in a session says `via: sso`.

## Workspaces

> **Experimental** - may change in any minor release, always noted in the changelog (what that means). A server with only `default` behaves exactly as before, and that part is under the promise.

A workspace is **delegated administration inside one server** (ADR 0023): one gateway for many teams, each managing its own connections, saved queries, API keys, roles, destinations and exports - each name unique within it - without reaching another's. It is not tenant isolation for a hosted service: every workspace runs in the same process, on the same store, with passwords under the same encryption key, and server-wide administrators see everything. It protects teams from each other's mistakes and reach, not from the people who run the server.

- `default` holds everything a server had before workspaces, and what is made without naming one. It can't be removed. A server that never creates a second workspace sees no difference.
- **Server-wide roles and memberships.** An administrator's own `role` applies everywhere, as before. A **membership** gives them one of the same four roles in one workspace and nothing outside it - without what only a server-wide role gives: managing administrators, the server's settings, re-encrypting secrets, applying configuration files, deleting a person's runs, and creating or removing workspaces. A workspace's owner manages its members. An administrator whose only roles are in other workspaces gets `403 workspace_required`, naming them.
- **Removing** a workspace takes it being empty: what it owns is deleted, or moved, first - never with it. Removing an administrator ends their memberships.
- Every change - a workspace created, described or removed, a member added, changed or removed - is in the audit log.

**Reaching a workspace.** Every route - `/q/<name>`, `/execute_sql`, `/catalog`, `/openapi.json`, the Management API

- also answers under `/w/<workspace>/...`, in that workspace: `/w/team-a/q/orders`, `/w/team-a/api/v1/queries`.
- **An API key** belongs to the workspace it was created in, and every call it makes is there - with or without the prefix. A signed-in user's role is found by name in the workspace the path names (`default` without one), so the users of another workspace call `/w/<workspace>/...`. MCP: a key's tools are its workspace's queries.
- **An administrator** acts in the workspace the path names, or `default`. One whose roles are all memberships names their workspace; unprefixed, they get `403 workspace_required`.
- **Naming a workspace you aren't in** - or one that doesn't exist - is `404 workspace_not_found`, either way, so a caller can't learn which workspaces there are.
- **Kept apart** in each workspace: its run history, usage quotas (two workspaces' keys of one name are two callers), cached responses and the cache's management, write requests, descriptions and configuration-file records. A local-folder destination is only in `default`: the folders exports may write under are the server's.
- **The Console**: once there is a second workspace, a switcher under the wordmark picks the one the tab works in - every screen then reads and changes that workspace's things - and the **Workspaces** screen lists them, creates and removes them (owners), and manages each one's members. An administrator whose roles are all in workspaces starts in their first.
- **Configuration files and the CLI**: a file's `workspace:` says where it applies (Configuration as code); `queryapigate export`, `exports run|list`, `collection export|import`, `config export` and `examples` take `--workspace NAME`.

**Seeing what happened, per workspace.**

- **The audit log**: each entry says where it happened - `workspace`, inside the hash chain - or `null` for the server's own (its administrators, sign-ins, re-keying). A member reads their workspace's entries; a server-wide role reads all of them, or one workspace's (`?workspace=`). Verifying the chain, its delivery to a SIEM and the list of instances are the server's: server-wide roles only.

- **Alerts** are the workspace's own - its keys, quotas, queries, connections, exports, writes waiting for approval. The server's alerts (an open server, the shared key used, SSO, encryption, the audit trail, history) are for server-wide roles.
- **Live events** (`/events`, and `queryapigate events` at `/w/<workspace>/events`) reach the subscribers of the workspace the run happened in.
- **`/metrics`** counts a key or connection outside `default` under its workspace-qualified name - `key="team-b/partner"`, `connection="team-b/lite"` - so two workspaces' `partner` keys are two series. `default`'s names are as they were.

## Authentication and permissions

`QUERYAPIGATE_API_KEY`, if set, is a full-access **admin** key - the break-glass key once named administrators exist (above) - unrestricted, exactly as before this section existed. Scoped keys are additive, managed through `/api/v1/api-keys` (admin only), and can only run queries: a list of connection names they may use (or every connection), and whether they may write at all. A scoped key can never do more than the server-wide settings already allow - `allow_writes` on a key can only narrow `QUERYAPIGATE_ALLOW_WRITES`, never widen it - and can never manage connections, saved queries or other API keys; only the admin key can. Creating your first scoped key turns on authentication for the whole server immediately, even without `QUERYAPIGATE_API_KEY` set - and since only the admin key can manage the server, doing that without also setting `QUERYAPIGATE_API_KEY` locks configuration changes out until you do (the server logs a warning at startup in that state).

A key's secret is never stored - only its SHA-256 hash, in `queryapigate.db` (`QUERYAPIGATE_HOME`). It is generated by the server and returned exactly once, when the key is created; there is no way to recover it afterwards, only to revoke it (`DELETE /api/v1/api-keys/<name>`) and create a new one.

`POST /api/v1/api-keys` creates a key:

```json
{"name": "reporting", "connections": ["reporting-db"], "allow_writes": false}
```

The `201` response is the key's grants and state plus its `secret` (`sk_...`), shown this once and sent with `Cache-Control: no-store`.

`connections` may be omitted (or `"*"`) for every connection, or an empty list to block all of them. `GET /api/v1/api-keys` lists keys (name, grants, active, expiry, created_at, `created_from_role`, `last_used_at` - never the hash or secret). Each entry also carries a live `usage` object - `{"queries": ..., "errors": ..., "rows": ...}` - the same live, in-process aggregation `GET /api/v1/connections` carries (see above), giving a key's activity alongside its grants; unlike a connection's, a key's `usage` never includes `avg_duration_ms` - the underlying latency histograms aren't split by key, to keep `/metrics`' bucketed output from growing with the number of keys (see Observability). `PATCH /api/v1/api-keys/reporting` changes any grant, `expires_at` or `active` (`false` revokes it immediately) without rotating the secret. `DELETE /api/v1/api-keys/reporting` removes it outright. Creating several keys with the same grants repeatedly? See Permission roles below for a reusable template - `POST /api/v1/api-keys` with `"role": "<name>"` instead of these fields.

### Per-saved-query access (external clients)

`connections` grants a key everything on a connection - every saved query on it, plus ad-hoc SQL if `allow_writes` and the server allow it. That fits an internal caller, but not an external one who should only ever reach a specific, curated list of saved queries and nothing else on the connection behind them.

A key's `queries` grant covers that case: a list of saved-query names it may run **regardless of `connections`**, independent of and additive with whatever `connections` already allows - never a narrower version of it. A key can have `connections: []` (no connection access at all) and still run every query named in `queries`, but it can never reach ad-hoc SQL through this grant, since `queries` only ever authorizes the specific named saved query, not the connection behind it - and only on **that query's own connection**: a request that names a different one with `?connection_name=` (or that supplies one for a query with no default) is `403` unless the key also holds a real grant on that connection. This applies equally to a collection grant.

```json
{"name": "acme-corp", "connections": [], "queries": ["monthly_revenue", "active_users"], "allow_writes": false}
```

`queries` may also be `"*"` for every saved query by name (still never ad-hoc SQL) - a middle tier between a single-connection key and a full-access one, for a caller that should see the whole curated catalogue but never write raw SQL. Omitted (or `[]`) grants nothing extra beyond `connections`, unchanged from before this field existed. `/openapi.json` and `/docs` reflect a key's actual reach: a `queries`-scoped key sees only its own approved queries in the catalogue, not the full internal list.

To grant a whole group instead of naming each query, see Granting access to a collection.

### Per-query write curation

A `queries` entry can be an object instead of a plain name, adding write access to that one query specifically - on top of, never instead of, whatever `allow_writes` already grants:

```json
{"name": "partner", "connections": [], "allow_writes": false,
 "queries": ["read_orders", {"name": "submit_order", "allow_writes": true}]}
```

Here `partner` can run `read_orders` read-only and `submit_order` (a write) - a single curated write endpoint - with no blanket write access, no connection access, and no ad-hoc SQL of any kind. A plain string entry stays exactly what it always was: read access only. `"*"` can never carry write access, by design - a key wanting write access to a specific query must enumerate its `queries` list explicitly rather than hiding a write grant behind a wildcard picked for unrelated read access. Still subject to the usual ceiling: never wider than server-wide `QUERYAPIGATE_ALLOW_WRITES`, and the query's SQL still has to pass the normal guard (a single statement, actually a write, and - if the key has `allowed_write_ops`

- one of the permitted keywords).

### Key expiry

A key can carry an optional `expires_at` (`YYYY-MM-DD`) for time-boxed access - a trial integration, a partner engagement with a known end date - that stops authenticating on its own once the date passes, without anyone having to remember to come back and revoke it:

```json
{"name": "trial-partner", "connections": ["reporting-db"], "expires_at": "2026-12-31"}
```

Valid through the *end* of that date (23:59:59), not from its start. Checked live on every request, the same way `active` already is - there is no background sweep, so nothing to schedule or fail silently. Omit it (or leave it unset) for a key that never expires; `PATCH /api/v1/api-keys/<name>` with `{"expires_at": null}` clears an existing expiry without rotating the secret, and `PATCH` without the field at all leaves whatever expiry (or lack of one) the key already had untouched.

### Last used

`GET /api/v1/api-keys` reports `last_used_at` for a key once it has authenticated at least one request - useful for noticing a stale key nobody has called in months (a candidate to revoke) or confirming a newly-issued one actually got wired up on the other end. Updated at most once a minute per key regardless of how often it's actually used, so a busy key doesn't turn every request into a disk write - read it as "roughly how recently," not an exact timestamp. A key that has never been used has no `last_used_at` field at all.

### Per-key rate limiting

`QUERYAPIGATE_RATE_LIMIT` (see Rate limiting and CORS) applies server-wide, by client IP, shared by every caller. Hand scoped keys to several external clients and they all draw from the same budget - one noisy integration can exhaust it for everyone else. A key's own `rate_limit` gives it an individual quota instead:

```json
{"name": "acme-corp", "connections": [], "queries": ["monthly_revenue"], "rate_limit": "100/minute"}
```

Same `N/period` grammar as `QUERYAPIGATE_RATE_LIMIT` (`second`, `minute`, `hour` or `day`). Checked **in addition to** the server-wide limit, never instead of it - a key can never use its own quota to exceed the ceiling every caller already sits under, and a per-key limit still applies even when `QUERYAPIGATE_RATE_LIMIT` is unset entirely, since throttling one specific external caller is a reasonable ask on its own. Omitted (or `null`) means no limit of this key's own - `PATCH /api/v1/api-keys/<name>` with an explicit `{"rate_limit": null}` clears an existing one, the same pattern `expires_at` uses.

### Per-key limits: concurrency, time and rows

A rate limit counts calls, not their cost: twenty slow queries a minute stay under `60/minute` and can still hold every connection in the pool. Three more grant fields, on keys and roles (ADR 0011), give one caller less than the server allows - each **narrows** the server's own limit, never widens it:

```json
{"name": "acme-corp", "queries": ["monthly_revenue"], "max_concurrent": 2, "query_timeout": 5,
"max_rows": 500}
```

| FIELD | WHAT IT LIMITS | EFFECTIVE |
|---|---|---|
| `max_concurrent` (1-1000) | queries this caller has running at once - from when one is sent until its last row is read, so a stream or a large export holds its slot as long as it runs; REST, ad-hoc SQL and MCP tools all count, a cached response and a live `/events` stream don't | over it: `429 too_many_concurrent`, `Retry-After: 1` - not queued |
| `query_timeout` (seconds, 0.1-86400) | how long one query may run | the smallest of this, the saved query version's own `timeout`, `?timeout=` and `QUERYAPIGATE_QUERY_TIMEOUT` |
| `max_rows` (1-10,000,000) | rows one call returns | the smaller of this and the server's limit for that kind of call: a page is cut to it (pages of `max_rows`, `X-Has-More`), a stream stops there, an MCP result is `truncated` |

- **With several instances**, `max_concurrent` is counted in Redis (`QUERYAPIGATE_REDIS_URL`), one count for all of them. Each running query holds a slot with a lease - its time limit plus 30 seconds, renewed while a stream runs - so a slot held by a process that died is freed when its lease ends. Without Redis, or while it's failing, each process counts its own, as rate limits do.
- **A value above the server's** is kept as granted, and the server's applies: the server's limit can change later. A key's `effective_limits` (`GET /api/v1/api-keys/{name}`) says what each comes to now.
- **Signed-in users** get their role's, counted per user. Administrators and the break-glass key have none.
- `null` clears one (`PATCH /api/v1/api-keys/{name}` with `{"max_rows": null}`), as for `rate_limit`.

## Usage quotas

> **Stable** since 0.30, under the compatibility promise (what that means). What counts as a call is fixed, as below; periods, counts and fields may still be added. Limits: a caller can overshoot by the calls it has in flight, and several instances without Redis count apart (*Precision*, below).

Partner and data-product contracts are written in quotas - "100,000 calls a month" - not per-minute rates. A key or role can carry one:

```json
{"name": "acme-corp", "queries": ["monthly_revenue"], "quota": {"calls": 100000, "rows": 10000000, "per": "month"}}
```

- **Per caller, per calendar period:** `per` is `day` or `month`, in the server's time zone. A key has its own counter; signed-in users get their role's quota **each**, counted per user. Either count may be left out, not both.
- **What counts:** successful calls that return data - saved queries, ad-hoc SQL, MCP tool calls (not `list_tables`), cached responses - and the rows they return (a page's `X-Row-Count`; a stream's rows as they leave, however it ends). An agent's write request counts as a call when it's made, with no rows. Refused calls don't: unauthorised, invalid, over a limit, or a database error. Nor does what isn't a caller asking for data: exports (they run as their schedule says), live events, MCP resources such as the data dictionary, and the Management API.
- **Used up:** `429 quota_exceeded`, naming which (`quota`: `calls` or `rows`), the `limit` and `resets_at`, with `Retry-After` until then. A cached response is refused too.

- **Every response to a caller with a quota** carries `X-Quota-Remaining-Calls` and/or `X-Quota-Remaining-Rows`, and `X-Quota-Reset` - so a client can slow down before it's refused.
- **Precision:** checked before a call, counted after it, so a caller can overshoot by the calls it has in flight at once; the call that crosses the row quota completes (its rows aren't known before it runs - `max_rows` bounds it). Counts are kept in the store - written in the run-history batch (`QUERYAPIGATE_HISTORY_FLUSH_INTERVAL`), not a write per call - and, **with several instances, counted in Redis** (`QUERYAPIGATE_REDIS_URL`), one counter for all of them, seeded from the store. Measured on two instances, 32 clients calling as fast as they could: 1,016 calls answered for a quota of 1,000. **Without Redis**, each instance sees the others' counts only once they're written and re-read: a burst shorter than that got 1,615 calls through for 1,000. One instance alone is exact.
- **Watching it:** a key's `usage_this_period` (`calls`, `rows`, their limits, `period_start`, `resets_at`); `GET /api/v1/usage/quotas` lists every caller's counts for this day and this month, for metering of your own (it's not billing); the `quota_nearly_used` alert fires at 80% and when used up. `null` clears a quota.

### IP allowlisting

A key can also be pinned to `allowed_ips`, a list of IP addresses or CIDR ranges (IPv4 or IPv6, mixed freely) it may authenticate from - real defense in depth for a key handed to an external party with known, stable infrastructure, since even a leaked key then only works from an expected address:

```json
{"name": "trial-partner", "connections": ["reporting-db"], "allowed_ips": ["203.0.113.5",
"198.51.100.0/24"]}
```

Checked against the same client address `QUERYAPIGATE_TRUST_PROXY` / `ProxyFix` already establish as trustworthy for rate limiting, not re-derived here - set `QUERYAPIGATE_TRUST_PROXY` correctly behind a reverse proxy, or every caller looks like the proxy's own address. This restricts *who* may use a key at all, independent of per-key rate limiting above, which restricts *how much* a caller who is already allowed may do. A request from an address outside the list fails exactly like a wrong key (`401`), not a distinct error - a caller learns nothing about *why* a key didn't work. Omitted (or `null`) means no restriction - the admin key is never restricted by this at all. `PATCH /api/v1/api-keys/<name>` with an explicit `{"allowed_ips": null}` clears an existing restriction, the same pattern `expires_at` and `rate_limit` use.

### Write operation granularity

A key with `allow_writes` on can be narrowed further with `allowed_write_ops`, a list of the specific SQL statement keywords it may actually perform - `INSERT` but not `DELETE` / `DROP`, for example - rather than every write keyword being equally permitted once writes are on at all:

```json
{"name": "ingest-bot", "connections": ["events-db"], "allow_writes": true, "allowed_write_ops":
["insert"]}
```

Only ever narrows write access, never widens it, and never restricts a read-only statement - a key with no `allow_writes` still can't write regardless of this list. Checked in `sqltools.validate_sql()` against the statement's own leading keyword (case-insensitive); a rejected statement gets `403` naming the operations the key *is* permitted. Omitted (or `null`) means every write keyword is equally permitted, exactly today's behaviour. `PATCH /api/v1/api-keys/<name>` with an explicit `{"allowed_write_ops": null}` clears an existing restriction, the same pattern `expires_at` / `rate_limit` / `allowed_ips` use.

### Table access restrictions

A key can be narrowed to a specific set of tables it may query, `allowed_tables` - unlike `allowed_write_ops`, this restricts *every* statement, read or write, since a table a caller shouldn't see is forbidden regardless of what's being done to it:

```json
{"name": "reporting-narrow", "connections": ["reporting-db"], "allowed_tables": ["orders", "customers"]}
```

Needs the `sqlglot` package to parse SQL (`pip install "queryapigate[flow]"` - the same optional extra the Access tab's "Query flow" diagram already uses; a query is rejected with `500` if it's missing while a key has `allowed_tables` set, rather than silently letting the query through unchecked). Checked against every table a statement actually touches - joins, subqueries and CTEs are all resolved correctly (a CTE's own name is never mistaken for a real table), including the target table of a bare `DELETE` / `UPDATE` / `INSERT`. Only supported for `mysql`, `postgres`, `clickhouse`, `sqlite` and `duckdb` connections, the dialects QueryAPIGate can actually parse for this - **a table-restricted key used against an `h2` or `jdbc` connection is refused on every query, with a `403` naming the unsupported connection type, never silently left unrestricted.** On a `mongo` connection the list is the collections the key may `find()` in - one outside it is refused with `403 table_not_allowed` before the server is asked anything (from 0.34.1; until then it wasn't checked on MongoDB at all).

Table names are matched case-insensitively and **qualified** (from 0.43; Trino's always were, as `catalog.schema.table`). A grant or a query's table that leaves out its schema means the connection's default:

| CONNECTION | `CUSTOMER` MEANS |
|---|---|
| `mysql`, `clickhouse` | the connection's `database`: `crm.customer` |
| `postgres` | `public.customer`, or the connection's `options.schema` instead of `public` |
| `sqlite`, `duckdb` | `main.customer` |

So `["customer"]` doesn't reach `billing.customer`: grant that as `billing.customer`. A query naming a catalog too (`shop.public.orders`) needs a grant written the same way. On PostgreSQL, a table-restricted call runs with `search_path` set to that default schema, so its unqualified names resolve where they were checked, whatever the role's own `search_path`. A table-valued function (e.g. ClickHouse's `numbers(10)`) touches no real table, so this can't meaningfully restrict one.

**Statements whose tables can't be checked are refused** for a table-restricted key, with `403 table_check_failed`: `SHOW`, `SET`, `USE`, `ATTACH`, `PRAGMA`, `CALL`, `PREPARE` / `EXECUTE`, `DECLARE` / `FETCH`, and any statement the parser keeps only as text. Each could read a table, or its columns, or reroute a later call's names, without naming it where the check looks. `EXPLAIN` is checked against the statement it explains. The key's view of the schema follows the same list: `GET /connections/<name>/schema` and MCP's `list_tables` leave other tables out (a foreign key pointing at one shows as `null`), and `table_ddl` answers a forbidden table with the same `404` as a table that doesn't exist.

Omitted (or `null`) means no restriction, exactly today's behaviour. `PATCH /api/v1/api-keys/<name>` with an explicit `{"allowed_tables": null}` clears an existing restriction, the same pattern the other grant fields use.

## Column masking

> **Stable** since 0.20, under the compatibility promise (what that means). Actions, and query shapes masking can follow, may still be added - a query refused today may be served masked later. A query served masked today is only ever refused instead by a security fix, if serving it could let a value out. Limits: callers with rules are refused on H2, JDBC and MongoDB connections, and masking hides values, not answers about them (below).

A `mask` grant - on an API key, a role (copied to keys made from it, and applied to signed-in users mapped to it), an export, or a **saved query itself** (`mask` in its definition, applied to every caller including administrators) - hides the values of named **source** columns in everything that caller receives (ADR 0005):

```json
{"mask": [{"column": "customer.email", "action": "hash"}, {"column": "*.phone", "action": "mask"}]}
```

- `column` is `table.column` or `schema.table.column`, with `*` wildcards.
- `action`: `hash` (HMAC-SHA256, 16 hex characters - equal values give equal codes; keyed with `QUERYAPIGATE_MASK_KEY`, or a key the store makes once), `mask` (text keeps its first character, an email its domain: `j**@example.com`; other values become null), `null`, `drop` (the column is removed), or `keep` (left as is on purpose - for a view, below). The strictest that applies wins.

**Traced, not matched by name.** Each result column is traced through the SQL to the table columns it is computed from - aliases, expressions, CTEs, subqueries, unions, joins, window functions - with the connection's schema to expand `SELECT *`; one computed from a masked column is masked as a whole (`lower(email)` is hashed). Applied on `/q/<name>` and `/execute_sql` in every format, paged or streamed, MCP tools and exports; a cached response is kept per set of rules. A query's own rules and the caller's are **unioned** - per column the strictest action wins - so a query-level mask can only ever tighten what a caller sees, never loosen it.

**Refused, never served unmasked** (`403 mask_unresolvable`, before the query runs): a result column that can't be traced - table functions, whole-row references (`SELECT c FROM customer c`), a table alias that renames columns, `SELECT *` over a `NATURAL` or `USING` join, `LATERAL`, recursive CTEs, DuckDB's `COLUMNS(...)`, functions masking can't see into - and a view column a rule might mean, until a rule names the view (`*.email`, `view.email`, or `view.email` with `keep`). A caller with rules is refused on H2, JDBC and MongoDB connections, and never gets the database's own error text. Needs `sqlglot` (`queryapigate[flow]`; `masking_unavailable` otherwise).

Each run's history entry lists what was masked (`masked`: column, action, source); `GET /catalog` shows a caller with rules which of each query's columns it gets masked (`masking`), without the source tables. Masking hides values in results - it doesn't stop a caller that writes SQL from filtering or grouping on a masked column; for that, give it saved queries only. The step by step: hide personal data from a partner or an AI agent.

## Permission roles (templates)

Creating several keys with the same shape of grants - the same connections, the same curated queries, the same rate limit - means repeating that shape by hand each time. A named role, managed through `/api/v1/roles` (admin only, stored separately from keys), is a reusable *template* for exactly that: `connections`, `allow_writes`, `queries`, `collections`, `rate_limit`, `allowed_ips`, `allowed_write_ops` and `allowed_tables`, the same fields a key itself carries (deliberately excluding `expires_at`, which is inherently per-key, not something a shared template should dictate).

```json
{"name": "reporting", "connections": ["reporting-db"], "allow_writes": false, "rate_limit": "200/hour"}
```

`POST /api/v1/api-keys` with `"role": "reporting"` instead of specifying grants directly copies that role's fields onto the new key **once, at creation time**:

```json
{"name": "acme-corp", "role": "reporting"}
```

This is a template, not a live link - a key created from a role is a fully independent copy from that moment on. `authenticate()` reads only the key's own stored entry on every request; the role is never consulted again. **Editing or deleting a role afterward has no effect whatsoever on a key already created from it** - there is no blast radius to updating a role once keys already exist from it, and no dangling reference to worry about when deleting one. A key still records which role (if any) it was created from, in `created_from_role` - purely informational, visible in `GET /api/v1/api-keys`, never consulted by any permission check.

`role` cannot be combined with any explicit grant field (`connections`, `allow_writes`, `queries`, `collections`, `rate_limit`, `allowed_ips`, `allowed_write_ops` or `allowed_tables`) in the same `POST /api/v1/api-keys` request - that combination is rejected with `400`, naming the conflicting fields. Create the key from the role, then `PATCH` it afterward to customize it away from the template. `expires_at` is the one field that *can* still be set alongside `role`, since it's per-key by nature rather than part of the shared template.

`GET /api/v1/roles` lists roles; `PATCH /api/v1/roles/<name>` updates one (the same explicit-null-to-clear convention as `PATCH /api/v1/api-keys/<name>` for `rate_limit`, `allowed_ips`, `allowed_write_ops` and `allowed_tables`); `DELETE /api/v1/roles/<name>` removes it - again, with zero effect on any key already created from it.

### Signed-in users (JWT)

An API key identifies an application. For an app whose *users* each sign in - a mobile app, a customer portal - QueryAPIGate can accept the token your app already has instead, so every user calls the API as themselves and nobody has to issue them keys:

```http
GET /q/my_orders
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
```

Turn it on with **one** way to verify tokens, and a role saying what signed-in users may do:

| VARIABLE | MEANING |
|---|---|
| `QUERYAPIGATE_JWT_JWKS_URL` | Your identity provider's signing keys (Auth0, Cognito, Firebase, Keycloak, Azure AD, ...): `https://.../.well-known/jwks.json`. Default algorithm RS256. Requires `QUERYAPIGATE_JWT_ISSUER` and `QUERYAPIGATE_JWT_AUDIENCE`. |
| `QUERYAPIGATE_JWT_SECRET` | Or: a shared secret, for tokens your own backend signs (HS256 by default; at least 32 bytes). |
| `QUERYAPIGATE_JWT_ISSUER` | The `iss` every token must carry. |
| `QUERYAPIGATE_JWT_AUDIENCE` | The `aud` value(s) accepted - this API's identifier at your provider (comma-separated for several). |
| `QUERYAPIGATE_JWT_ALGORITHMS` | Signature algorithms accepted, if not the default. |
| `QUERYAPIGATE_JWT_ROLE` | The role every signed-in user gets. |
| `QUERYAPIGATE_JWT_ROLE_CLAIM` | Or: a claim naming each user's role (a string or a list - the first existing role wins; dotted paths such as `app_metadata.roles` work). Falls back to `QUERYAPIGATE_JWT_ROLE`. |
| `QUERYAPIGATE_JWT_USER_CLAIM` | The claim identifying the user (default `sub`). |
| `QUERYAPIGATE_JWT_WORKSPACE_CLAIM` | A claim naming the workspace a user belongs to (experimental). The token, not the request path, decides a signed-in user's workspace, so a user signed in for one team can't reach another's by changing the `/w/<workspace>/` prefix. Unset, every signed-in user acts in the default workspace. |

Needs `pip install "queryapigate[jwt]"` (included in the Docker image). Settings that would make checking unsafe stop startup: both or neither of a secret and a JWKS URL, a JWKS URL without issuer and audience (a provider signs tokens for every app it serves) or over plain `http://`, a short secret, mixing HMAC and public-key algorithms (the classic algorithm-confusion attack), or no role.

**What is checked.** The signature; `exp` (required) and `nbf` / `iat`, with 30 seconds of leeway for clock skew; `iss` and `aud` when set; and only the configured algorithms - never `none`. A failure is a plain `401`, exactly like a wrong API key. Signing keys are fetched from the JWKS URL once and cached, and fetched again only for a key id not seen before - a provider rotating its keys just works.

**What a user may do** is their role's grants - connections, queries, collections, writes, tables, `allowed_ips`, and `rate_limit`, counted per user. Unlike a key created from a role, which copies the role once, signed-in users use the role *live*: editing or deleting it applies on their next request. A token whose role doesn't exist is refused. When JWT is on, the server never runs in open-access mode: a request with neither a key nor a token is refused even if no API key is configured.

**Who they are.** The caller's name is `jwt:<user claim>` - in logs, run history (filter with `GET /api/v1/history?key=jwt:alice`) and live events, so each user's event stream carries only their own runs. (`/metrics` counts all signed-in users under one `jwt` label, so a large user base can't multiply its series.) A request carrying `X-API-Key` is judged on that key alone.

**Their own rows, guaranteed.** A parameter with `from_claim` takes its value from the verified token, so a query can only ever see the caller's data - there is nothing in the request to change:

```json
{"filename": "my_orders",
 "sql_query": "SELECT * FROM orders WHERE customer_id = :customer_id ORDER BY created_at DESC",
 "query_parameters": {"customer_id": {"type": "str", "from_claim": "sub"}}}
```

- A signed-in user who sends `customer_id` anyway gets `400`; a token without the claim gets `403`.
- The admin key has no token and supplies it like any other parameter (for testing). Any other API key - and any MCP client using one - gets `403`: it has no token to take the value from.
- It is left out of `/openapi.json`, `/catalog`, Postman exports and MCP tool schemas: callers never send it.

**Every query, not just this one: `require_claim_binding`.** The guarantee above holds for a query that binds the user; a query granted to the same role without a `from_claim` parameter shows every user every row it returns. For a role whose users are your customers, say so on the role, and the server holds every query to it:

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/roles/customers -H "X-API-Key: $TOKEN" \
  -H 'Content-Type: application/json' -d '{"require_claim_binding": true}'
```

- Its users run only saved queries with at least one `from_claim` parameter; another is `403 claim_binding_required`, and is left out of their `/catalog`, `/openapi.json` and MCP tools.
- They reach queries by name or collection only - never a connection, so no ad-hoc SQL, `find()` or schema browsing. Such a role has `connections: []` (left out, that's what it means here; another value is refused).
- It's a role's alone: a key created from the role doesn't take it - a key has no token to bind. In a configuration file, `require_claim_binding: true` on the role.
- Default off: a role without it works as it always has, for staff tools that do read every row.

A JWT can't be revoked before it expires, so keep token lifetimes short (minutes, refreshed by your app) - as identity providers do by default. Disabling a user's access at once means taking the query or the role away.

## Security headers

Every response sets these itself, so none depends on a reverse proxy to add them (ADR 0010):

| HEADER | VALUE | ON |
| --- | --- | --- |
| `X-Content-Type-Options` | `nosniff` | every response |
| `Content-Security-Policy` | `default-src 'none'; frame-ancestors 'none'` | API responses (JSON, CSV, Parquet, …): rendered by a browser, they run nothing and can't be framed. The Console and `/docs` have their own policy |
| `Referrer-Policy` | `no-referrer` (`same-origin` on the Console and `/docs`) | every response |
| `Cache-Control` | `no-store`, unless the route says otherwise: `private, max-age=<cache_ttl>` for a cached saved query, `no-cache` for live events, the Console's assets their own | every response |
| `Strict-Transport-Security` | `max-age=<QUERYAPIGATE_HSTS_MAX_AGE>` | only when that is set, and only on a request that arrived over HTTPS - behind a proxy, as `QUERYAPIGATE_TRUST_PROXY` lets the server tell |

The MCP server and `queryapigate events` send the same, but HSTS. `Server` carries no version (`QueryAPIGate` from `queryapigate serve`; gunicorn's is `gunicorn`; the MCP server sends none). `/docs` loads one exact Swagger UI release, each file with its Subresource Integrity hash, and its CSP allows exactly those files and its own script - a changed file on the CDN is refused by the browser, rather than run on the page where people type their API key.

## Observability

Every response carries `X-Request-Id` (12 hex characters by default); log lines written while handling that request carry the same ID and the name of the API key that made it (`admin` for `QUERYAPIGATE_API_KEY`, a scoped key's own name, or `-` when no key is configured at all), so a request - and who made it - can be traced through the logs even under concurrent traffic. A caller can supply its own ID by sending `X-Request-Id` - 1 to 64 characters of letters, digits, `.`, `_`, `:` and `-` (a UUID qualifies) - and that ID is then used everywhere instead: the response header, the log lines and a saved query's run history, so a run can be tied to a trace in the caller's own system. Anything else (too long, spaces, quotes, non-ASCII) is ignored rather than rejected, and the response header shows the ID actually used. It is a correlation aid only - nothing authorises by it, and two requests may share one if the caller sends the same value twice. Browsers can send and read it cross-origin (`X-Request-Id` is in the CORS allowed and exposed headers). Plain text by default; `QUERYAPIGATE_JSON_LOGS=1` switches to one JSON object per line (`time`, `level`, `logger`, `request_id`, `key`, `message`). In JSON mode, several log lines also carry extra structured fields alongside `message` rather than only inside it - the per-query line (`connection`, `dialect`, `limit`, `offset`, `timeout`, `sql_hash`), the streaming-start line (`connection`, `dialect`, `sql_hash`), the slow-query warning (`connection`, `dialect`, `duration_ms`), and the per-request access log line (`method`, `path`, `status`, `duration_ms`, and `serialization_ms` when the response went through the paged JSON/CSV/TSV/XML/YAML/XLSX/Parquet formatter) - so a log aggregator can filter or aggregate on those directly instead of parsing the message text. `sql_hash` is a full SHA-256 hex digest of the SQL text, logged *alongside* the full text (never instead of it) - useful for spotting "did this same query run elsewhere/before" without a log aggregator having to store or search the SQL itself.

A query that takes at least `QUERYAPIGATE_SLOW_QUERY_THRESHOLD` seconds (default 1; `0` disables it) is logged as a `WARNING` with the connection, dialect and elapsed time.

A saved query's `execution_history` entries (see Save a query) also carry `request_id`, `key_name` and `serialization_ms`, so a slow or failed run visible in the admin UI's History tab can be traced back to the exact structured log line (and caller) that produced it, and so response-formatting time can be told apart from `duration_ms` (query execution time) - useful for XLSX or other large-page exports, where encoding cost can rival query time but was previously invisible, folded into "whatever's left over" between total request latency and query latency.

`GET /metrics` serves Prometheus text exposition format. It's open to whoever can reach it unless **`QUERYAPIGATE_METRICS_TOKEN`** is set (at least 16 characters): then it answers `401` to anyone but a scraper sending `Authorization: Bearer <the token>` and an administrator whose role may observe - the Console's Metrics screen. Set it wherever the server is reachable from outside: the labels name your API keys, queries and connections. The Helm chart sets one (`metricsAuth`). Administrators are counted under one `admin` label, and signed-in users under one `jwt`, so no person's name is a label.

- `queryapigate_requests_total` - HTTP requests by method, endpoint, status and the calling key's name. `queryapigate_request_duration_seconds` - the same latency, by method and endpoint only (not by key, to keep the bucketed output from growing with the number of keys).
- `queryapigate_queries_total` - SQL queries by connection, dialect, status (`success` / `error`) and the calling key's name. `queryapigate_query_duration_seconds` - the same latency, by connection and dialect only.
- `queryapigate_rows_returned_total` - total rows actually returned, by connection, dialect and the calling key's name: the trimmed page for a paged query, or however many rows made it out of a streaming export before it finished or failed partway through (a partial count on failure is still counted - that data already left the server).
- `queryapigate_active_queries` - a gauge of SQL queries currently executing right now, paged or mid-stream. For a streaming export this stays incremented for as long as the client keeps reading, not just for the initial query dispatch, since the underlying connection stays checked out the whole time.
- `queryapigate_serialization_duration_seconds` - response body serialization latency (JSON/CSV/TSV/XML/YAML/ XLSX encoding), by output format, for paged responses only - streaming formats row by row as it goes, so there's no equivalent single span to measure there.
- `queryapigate_pool_idle_connections` - idle pooled database connections currently held.
- `queryapigate_rate_limit_rejections_total` - requests rejected by the rate limiter.
- `queryapigate_rate_limit_fallbacks_total` - rate-limit checks counted in this process instead of Redis, because Redis failed (see Rate limiting and CORS).
- `queryapigate_cache_hits_total` / `queryapigate_cache_misses_total` - responses served from, or missed in, the `cache_ttl` response cache (in-process by default, Redis-backed when `QUERYAPIGATE_REDIS_URL` is set - see `/api/v1/settings`). Counted from the same `X-Cache: HIT` / `MISS` header a cacheable response already carries.
- `queryapigate_cache_entries` - responses currently held in the response cache.

Metrics are kept in memory for this one process - the standard Prometheus model: with several instances, scrape each one and aggregate in queries (the bundled dashboard sums them). That's also why the image runs one gunicorn worker per container: a scraper reaching a container with several workers would get one of them at random. Scale out with more containers (Scaling out).

### Seeing it: a built-in view, or a real dashboard

The admin UI's **Metrics** tab reads `/metrics` itself and renders it as stat tiles, a couple of bar charts (requests by status, queries by connection) and a per-connection table (queries, errors, average latency, rows) - a zero-setup live view for a deployment with no Prometheus/Grafana stack in front of it at all. It's deliberately a *snapshot*, not a dashboard: the numbers are this process's own totals since it started, with no history and no trends, the same limits

above already describe. It reads `/metrics` as the administrator signed in to the Console, so it works with `QUERYAPIGATE_METRICS_TOKEN` set too; reading it again just re-fetches the current numbers - there's no polling or auto-refresh.

For real history, trends and alerting, scrape `/metrics` with Prometheus and import `documentation/grafana-dashboard.json` into Grafana (*Dashboards → New → Import*, then upload the file or paste its contents) - it builds on exactly the metric names listed above, with panels for request/query rate and latency (p50/p95/p99), error rate, rows returned, active queries, pool occupancy and rate-limit rejections. Each QueryAPIGate process needs its own scrape target, and the dashboard sums across them.

## Semantic search

> **Experimental** - the embeddings-endpoint settings and the `embed` parameter rule may change as they meet more vector stores. Postgres/`pgvector` only for now, and off unless `QUERYAPIGATE_EMBEDDING_URL` is set (what experimental means, ADR 0025).

A saved query can take **text from the caller and search by meaning**, not just by exact match. Mark a text parameter `"embed": true`: the caller passes words, and the server turns them into a vector (an *embedding*) and binds that vector, so the query's SQL can rank rows by `pgvector` similarity. The embedding is computed on the server, so a caller - or an AI agent - only ever supplies text; it never sees or chooses the vector, exactly as a `from_claim` value is the server's to set.

```json
{"filename": "find_articles", "connection_name": "docs_pg",
 "sql_query": "SELECT id, title FROM article ORDER BY embedding <=> CAST(:q AS vector) LIMIT :k",
 "query_parameters": {"q": {"type": "str", "embed": true, "description": "What to search for, in your
own words",
                            "example": "comfortable running shoes"},
                     "k": {"type": "int", "default": 5, "max": 50}}}
```

Turn it on with an OpenAI-compatible embeddings endpoint:

| SETTING | MEANING |
|---|---|
| `QUERYAPIGATE_EMBEDDING_URL` | The embeddings endpoint - OpenAI's (`https://api.openai.com/v1/embeddings`), Azure OpenAI's, or a local Ollama's (`http://localhost:11434/v1/embeddings`). |
| `QUERYAPIGATE_EMBEDDING_MODEL` | The model name, e.g. `text-embedding-3-small` or (Ollama) `nomic-embed-text`. |
| `QUERYAPIGATE_EMBEDDING_API_KEY` | A bearer token for the endpoint, if it needs one. Omit for a local Ollama. |

The same governance applies to a semantic query as any other: column masking, row-level `from_claim` filtering, rate limits and the audit log all still hold - so an agent can search your data by meaning while still seeing only what it's allowed to. A query with an `embed` parameter is refused with `embedding_unavailable` when no endpoint is configured, and with `unsupported_operation` on a non-Postgres connection - never run with the raw text bound where a vector belongs.

# Tracing (OpenTelemetry)

> **Experimental** - new in 0.23 (ADR 0010); span names and attributes may change in any minor release, always noted in the changelog, as OpenTelemetry's conventions for databases and AI agents settle. What never goes into a span won't change.

Spans to your tracing backend (Jaeger, Tempo, Honeycomb, Datadog…) through OTLP, so the time between your app and its database shows where it went - authentication, the query, serialising the result. Install the extra and set an endpoint, the standard way:

```shell
pip install "queryapigate[otel]"
OTEL_EXPORTER_OTLP_ENDPOINT=http://collector:4318 queryapigate serve
```

Every other OpenTelemetry variable works as for any service: `OTEL_EXPORTER_OTLP_TRACES_ENDPOINT`, `OTEL_SERVICE_NAME` (default `queryapigate`), `OTEL_RESOURCE_ATTRIBUTES`, `OTEL_TRACES_SAMPLER`, the exporter's `..._HEADERS` and `..._TIMEOUT`, `OTEL_SDK_DISABLED`. An endpoint set without the extra stops the server, saying what to install. Spans are exported in the background: a collector that's down costs spans, never a slow request. Each process - `serve`, `mcp`, `events`, an export - is its own `service.instance.id`, the id the Instances screen shows.

| SPAN | KIND | WHAT IT RECORDS |
|---|---|---|
| `GET /q/<name>` - the route rule | server | `http.request.method`, `http.route`, `http.response.status_code`, `url.scheme`, `client.address`, `queryapigate.request_id`, `queryapigate.caller`; `error.type` (the error `code`) on a 4xx or 5xx |
| `query <name>` | internal | `queryapigate.query`, `.version`, `.connection`, `.rows`, `.masked`, `.cache` (`hit`, `miss`); `.agent_write` for an agent's write |
| `SELECT <connection>` (the statement's kind) | client | `db.system.name`, `db.operation.name`, `db.namespace` (a network database's name), `server.address`, `server.port`, `db.response.returned_rows` or `db.response.affected_rows` |
| `serialize <format>` | internal | `queryapigate.format`, `queryapigate.bytes` |
| `tools/call <tool>` (MCP) | server | `gen_ai.tool.name`, `queryapigate.caller`; `error.type` when the tool failed |
| `GET /events` | server | opening a live stream - one span, not one per event |
| `export <name>` | internal | a saved export or `queryapigate export`: `queryapigate.rows`, `queryapigate.bytes` |

- **Never in a span:** parameter values, rows or anything from them, statement text, query strings, headers (keys, tokens, cookies), connection passwords, write requests' arguments - a test fails if any does. Spans go to a collector whose readers have no data access. Exceptions aren't recorded either, since a database's message can carry values: a failed span has `error.type`, the error's code or class.

- **A caller's `traceparent` is continued** on REST and MCP requests, so QueryAPIGate's spans sit inside the caller's trace; `queryapigate export` continues `TRACEPARENT` from its environment, for a scheduler that sets it. Whether a trace is kept follows `OTEL_TRACES_SAMPLER`: by default the caller's sampled flag decides - set `traceidratio` so callers don't.
- **The trace id goes where the request id goes:** `trace_id` in run history entries, in error bodies beside `request_id`, and in JSON log lines (with `span_id`).
- A streamed result's database span covers opening the stream, up to its first row, not the whole transfer.

## Live events (Server-Sent Events)

> **Stable** since 0.34, under the compatibility promise (what that means), with the payload the event log brought in 0.33 (ADR 0021). Event types, payload fields, filters and settings may still be added - skip a type or field you don't know; ids, the fields below and what each filter means stay as they are. Limits: an event is kept for `QUERYAPIGATE_EVENTS_RETENTION_HOURS`, and a resume from further back starts at the oldest still kept; a run dropped under back-pressure, before it was written, is no event; a revoked key's or expired token's stream is closed within about a minute and a quarter, not at once; a key sees no event from before it was created; one key may hold as many of `queryapigate events`' streams as it likes, up to `QUERYAPIGATE_EVENTS_MAX_CONNECTIONS`; the main server's own `/events` has no ids or resume, and sees only its own process.

Every run - of a saved query, or of ad-hoc SQL - can be streamed to clients as it happens: a mobile app showing a user their own requests completing, a dashboard, the Console's Home. There are two ways to serve that stream, with the same format:

| | `GET /EVENTS` ON THE MAIN SERVER | `QUERYAPIGATE EVENTS` |
|---|---|---|
| For | the Console, a few clients | apps and phones - many clients |
| Open streams | `QUERYAPIGATE_EVENTS_MAX_STREAMS` (default 4) - each holds a request thread | thousands per process (`QUERYAPIGATE_EVENTS_MAX_CONNECTIONS`, default 10,000) |
| Sees runs from | this process only | every worker and instance sharing the store |
| Event ids and resume (`Last-Event-ID`) | no | yes, for `QUERYAPIGATE_EVENTS_RETENTION_HOURS` (default 24) |
| Delay | immediate | up to `QUERYAPIGATE_HISTORY_FLUSH_INTERVAL` (default 1 s), plus `QUERYAPIGATE_EVENTS_POLL_INTERVAL` on SQLite |

**Each key gets its own view, not a shared firehose.** Either way, a real API key is required (unless the server runs with no keys at all). The admin key sees every event; any other key sees only the runs made with that same key since it was created - its own personal activity feed. (A deleted key's name can be given to a new key; the old one's runs don't come with it.) A stream looks like this:

```text
GET /events
X-API-Key: <a scoped key's own secret>

id: 4812
data: {"type": "query.run", "source": "rest", "occurred_at": "2026-10-13 12:00:00", "kind": "saved",
       "query": "monthly_revenue", "version": 3, "connection_name": "warehouse",
       "entry": {"executed_at": "2026-10-13 12:00:00", "status": "success", "rows": 42, "duration_ms":
118.4,
                 "key_name": "mobile-alice", "request_id": "a1b2c3d4e5f6", "transport": "rest"}}
```

| FIELD | |
|---|---|
| `type` | what happened: `query.run` for every run. More types will be added - ignore one you don't know |
| `source` | where it came from: for a run, the front door - `rest`, `mcp` or `export`; `null` for a run that names none |
| `occurred_at` | when, in the server's time zone, to the second |
| `kind`, `query`, `version` | `saved` with the query and version that ran, or `adhoc` with both `null` - the names run history uses |
| `connection_name` | the connection it ran on |
| `entry` | the run's history entry. An ad-hoc run's carries its SQL as `QUERYAPIGATE_HISTORY_ADHOC_SQL` keeps it |

**Filters.** Ask for less with `?type=`, `?kind=` (`saved` or `adhoc`), `?query=`, `?connection=` and `?status=` (`success` or `error`), on either endpoint. Each takes one value, and they combine: `/events?query=monthly_revenue&status=error` is that query's failures. They narrow what the key may see, never widen it, and a resume applies them too. A `kind` or `status` that can never match is refused with `400 invalid_filter`; a query or connection that doesn't exist yet simply matches nothing.

**Event ids** are opaque integers that only increase, and are never handed out twice: store the last one and send it back, but don't read anything else into it (ADR 0002). Until 0.33 an id was a run's history row id; the event log continued that sequence, so an id held across the upgrade still resumes.

A connection with nothing to say sends a `: keepalive` comment line every 15 seconds so a proxy or client library doesn't time it out as idle - not an event, safe to ignore.

**Send the key in `X-API-Key` - or a signed-in user's token as `Authorization: Bearer` - never in the URL** (where it would leak into access logs and browser history). `queryapigate events` closes a stream once its token expires (checked every minute): reconnect with a fresh token and `Last-Event-ID`. A browser's `EventSource` can't set request headers, so in a browser read the stream with `fetch()` and a streamed response body; every mobile platform's own HTTP client can read a streamed response the same way.

### `queryapigate events`

A separate process, run beside `queryapigate serve` against the same `QUERYAPIGATE_HOME` or `QUERYAPIGATE_DATABASE_URL`, serving `GET /events`, `GET /health` and `GET /metrics` on its own port:

```shell
queryapigate events --host 0.0.0.0 --port 5002
```

Put it behind the same reverse proxy as the main server, routing `/events` to it - see DEPLOYMENT.md - so clients keep one base URL.

- **Every instance's runs.** Events come from the event log in the store, not from one process's memory. Every run is written there in the same batch as its history row, and an event's `id:` is its id there. On PostgreSQL the server is notified the moment a batch is written; on SQLite it checks every `QUERYAPIGATE_EVENTS_POLL_INTERVAL` seconds (default 1). So delivery takes up to one `QUERYAPIGATE_HISTORY_FLUSH_INTERVAL` - lower that (e.g. `0.2`) for snappier events.
- **Resume.** Reconnect with a `Last-Event-ID` header (or `?last_event_id=` if the client can't set headers) and the stream first replays every event after that id that the key may see and asked for, then continues live, with nothing missed or repeated in between. `EventSource` sends the header by itself; with `fetch()`, remember the last `id:` you read and send it when you reconnect.
- **How far back.** The event log keeps each event for `QUERYAPIGATE_EVENTS_RETENTION_HOURS` (default 24), whatever run history keeps: `QUERYAPIGATE_HISTORY_SAMPLE_RATE`, `QUERYAPIGATE_HISTORY_LIMIT` and retention apply to history only. A client away for longer gets the stream from the oldest event still kept. One row per run: on SQLite under heavy traffic, a shorter window keeps the store small. Runs dropped under back-pressure (see Run history) are lost to both.
- **Limits.** A stream whose key is revoked, deactivated or expires is closed within about a minute (at most 75 seconds: the key is checked again every 60, at the stream's next write). A client that stops reading is disconnected once 1,000 events are waiting for it - it reconnects with `Last-Event-ID` and catches up. Beyond `QUERYAPIGATE_EVENTS_MAX_CONNECTIONS` open streams, new ones get `503` with `Retry-After`; `QUERYAPIGATE_RATE_LIMIT` applies to connection attempts; `QUERYAPIGATE_CORS_ORIGINS` and `QUERYAPIGATE_TRUST_PROXY` mean the same as on the main server.
- **Metrics.** `GET /metrics` (behind `QUERYAPIGATE_METRICS_TOKEN`, like the main server's) counts this process's open streams (`queryapigate_events_streams`), events delivered, replayed on a resume, and clients cut off for falling behind (`queryapigate_events_delivered_total`, `_replayed_total`, `_slow_disconnects_total`), and how long an event took from happening to delivery (`queryapigate_events_lag_seconds`, to the second).
- **Scale.** Measured on a 4-core machine shared with PostgreSQL and the load generator: 9,000 concurrent streams in one process (about 170 MB), every event reaching all 9,000 within 0.75 s of the request, with the main server's own latency unaffected.

### `GET /events` on the main server

Immediate and needs no extra process, but each open stream occupies one of the main server's request threads for as long as it is open. So at most `QUERYAPIGATE_EVENTS_MAX_STREAMS` (default 4, of the Docker image's 8 threads) are held at once; beyond that it answers `503` with `Retry-After`, rather than leave no threads for requests (`0` turns it off). It only sees runs handled by its own process, and has no event ids or resume. A client that disconnects frees its slot at the next keepalive, within 15 seconds.

# Alerts

> **Stable** since 0.20, under the compatibility promise (what that means). New kinds and fields may still be added - a client should show, or ignore, a `kind` it doesn't know; none is removed or renamed except through the deprecation process.

`GET /api/v1/alerts` (admin only) lists what needs attention now, most severe first. The Console shows the same list on its Alerts screen, as a count on the bell in its header, and in Home's System health. Every alert is a live condition, worked out afresh on each request - nothing is stored, and an alert disappears by itself once its cause does (a key extended, a connection fixed, a query sped up).

```json
{"items": [{"id": "key_expiring:partner", "severity": "warning", "kind": "key_expiring",
            "title": "API key 'partner' expires in 3 days",
            "detail": "On 2026-10-07. Calls with it will be refused from then on - extend it, or give
its callers a new key first.",
            "since": "2026-10-07", "target": {"type": "key", "name": "partner"}}],
 "checked_at": "2026-10-04 12:47:00"}
```

`id` (kind and subject) stays the same while the condition holds, so a client can remember which alerts it has already reported or dismissed. `target` names what to fix: a `key`, `query`, `connection`, or a `settings` section.

| KIND | SEVERITY | WHEN |
|---|---|---|
| `open_server` | critical | No API key is configured: anyone can use and change the server |
| `connection_failing` | critical | A connection's last 3 runs in 24 hours all failed for a connection reason (`connection_failed`, `driver_missing`, …), not bad SQL |
| `key_expired` | warning | An active key's `expires_at` has passed |
| `key_expiring` | warning | An active key expires within 7 days |
| `query_errors` | warning | At least `QUERYAPIGATE_ALERT_ERROR_RATE` % (default 20) of a saved query's last 50 runs in 7 days failed, with 10 runs or more |
| `query_timeouts` | warning | A saved query timed out (`query_timeout`) in the last 24 hours |
| `query_slow` | warning | A saved query's median successful run (of its last 50 in 7 days, 5 or more) is over `QUERYAPIGATE_SLOW_QUERY_THRESHOLD` - typically slow, not slow once |
| `key_rate_limited` | warning | A key's own `rate_limit` refused it 10 times or more in the last hour |
| `client_rate_limited` | warning | `QUERYAPIGATE_RATE_LIMIT` refused one client address 10 times or more in the last hour |
| `history_failed`, `history_dropped` | warning | Runs could not be recorded, or were dropped because the store could not keep up |
| `instances_not_shared` | warning | Several processes share this store, and some of them without Redis - their rate limits and cache are per process |
| `instances_versions_differ` | warning | Processes sharing this store run different versions - expected only during a rolling upgrade |
| `break_glass_used` | warning | The shared `QUERYAPIGATE_API_KEY` was used in the last 24 hours although named owners exist |
| `export_failing` | warning | An export's newest run failed - nothing new reaches its destination until one succeeds |
| `rate_limits_not_shared` | warning | Redis failed a rate-limit check in the last ten minutes, so this process is counting limits on its own |
| `key_unused` | info | An active key unused for `QUERYAPIGATE_ALERT_KEY_UNUSED_DAYS` (default 90) |
| `encryption_key_rotation_incomplete` | warning, info | `QUERYAPIGATE_SECRET_KEY` lists several keys: passwords are still under an old one (warning - run `queryapigate rekey`), or none are and the old keys can be removed (info) |
| `audit_delivery_behind` | warning, critical | An audit log output to a SIEM is more than five minutes behind (critical while delivery is also failing); the entries wait, past the rolling cap |
| `secret_provider_failing` | critical | A secret provider - a file, Vault, AWS - has failed for five minutes in this instance; connections taking a secret from it work only while what it gave before is within its time |
| `audit_chain_broken` | critical | The audit log's hash chain doesn't verify: an entry was changed, removed, inserted or reordered (checked every 5 minutes) |
| `quota_nearly_used` | warning, critical | An API key has used 80% of a usage quota this period (warning), or all of it - its calls are refused until it resets (critical) |

| KIND | SEVERITY | WHEN |
|---|---|---|
| `writes_awaiting_approval` | warning | AI agents' writes are waiting for a person to approve them; each expires unapproved after `QUERYAPIGATE_WRITE_APPROVAL_MINUTES` |
| `sso_unreachable` | critical | Console sign-in through SSO is on, but the identity provider's metadata can't be fetched - nobody can sign in that way; open sessions keep working |
| `agent_unusual_volume` | warning | An AI agent read at least `QUERYAPIGATE_ALERT_AGENT_ROWS` rows over MCP in the last hour, and ten times its own average hour - see Agent usage |

| VARIABLE | DEFAULT | EFFECT |
|---|---|---|
| `QUERYAPIGATE_ALERT_ERROR_RATE` | `20` | Percentage of a saved query's recent runs that may fail before `query_errors`. `0` turns it off. |
| `QUERYAPIGATE_ALERT_KEY_UNUSED_DAYS` | `90` | Days an active key may go unused before `key_unused`. `0` turns it off. |
| `QUERYAPIGATE_ALERT_AGENT_ROWS` | `10000` | Rows an agent may read over MCP in an hour before a sudden rise is `agent_unusual_volume`. `0` turns it off. |

The history checks read the newest 5,000 runs, so their cost doesn't grow with how much history is kept. Rate-limit and history-write counts are this process's own, since it started (like `/metrics`): with several instances, each reports its share. For production paging, alert on `/metrics` from Prometheus too - this list is for the people running QueryAPIGate day to day, not a replacement for an on-call alerting system.

## Agent usage

> **Stable** since 0.30, under the compatibility promise (what that means). Fields per agent, and kinds of `refusals` and `writes`, may still be added - a client should ignore one it doesn't know. Limits: it holds what run history holds (below) - calls refused before they ran aren't in it, a sampled history is a sample, and one summary reads at most 100,000 runs.

What each AI agent did over MCP - for "what is this agent asking, and is it trying things it shouldn't?":

```shell
curl 'http://127.0.0.1:5000/api/v1/usage/agents?since=2026-10-10' -H "X-API-Key: $TOKEN"
```

```json
{"since": "2026-10-10", "until": "2026-10-11 10:42:07", "sample_rate": 1.0, "truncated": false, "items":
[
  {"agent": "jwt:7222e50f-b640-4e00-ab30-da4c0c9c9261", "calls": 41, "rows": 1830, "errors": 3,
"masked_calls": 12,
   "first_seen": "2026-10-10 09:12:44", "last_seen": "2026-10-11 10:40:03", "connections": ["examples"],
   "queries": {"example_top_films": 20}, "adhoc_calls": 21,
   "refusals": {"mask_unresolvable": 2, "table_not_allowed": 1},
   "top_statements": [{"sql": "SELECT first_name, email FROM customer LIMIT 2", "sql_sha256": null,
                       "count": 9, "rows": 18, "errors": 0}, ...]}]}
```

- An **agent** is a caller on the MCP server: an API key, or a signed-in user (`jwt:<sub>`). `since` (inclusive) and `until` (exclusive) take a date or a time, in the server's time; or `hours=6` for the last six hours. The default is the last 24 hours.
- **`refusals`** counts failures by error code: `table_not_allowed`, `mask_unresolvable` or `read_only` are an agent trying what its grant doesn't allow. **`masked_calls`** are the calls masking changed.
- **`writes`** counts its write requests over MCP (write tools) by how they ended - pending, approved, done, failed, rejected, expired; an agent with requests and no runs is listed too.
- **`top_statements`** are its most frequent ad-hoc statements, whitespace aside - by text, or by `sql_sha256` when `QUERYAPIGATE_HISTORY_ADHOC_SQL` is `hash` (none with `none`).
- It's worked out from run history, so it holds what history holds: calls refused before they ran (no credential, a rate limit) are only in `/metrics`; with `QUERYAPIGATE_HISTORY_SAMPLE_RATE` below 1, successful runs are a sample (`sample_rate` says so; failures are always kept); and at most 100,000 runs are read for one summary (`truncated`).
- **`agent_unusual_volume`** (Alerts) warns when an agent reads at least `QUERYAPIGATE_ALERT_AGENT_ROWS` (default 10,000) rows in an hour, and ten times its own average hour over the runs the alerts read.
- Who may: every administrator role (`observe`). In the Console: **Observability → Agents**.

## Instances

`GET /api/v1/instances` (admin only) lists the processes using this store now - every `queryapigate serve` worker, `mcp` and `events` process seen in the last 90 seconds - and what looks wrong about them:

```json
{"items": [{"id": "3f9c2a1b7d4e-4242", "host": "api-1", "pid": 4242, "role": "serve", "version":
"0.29.0",
            "shared_limits": true, "started_at": "2026-10-04 09:00:12", "last_seen": "2026-10-04
12:47:03",
            "this": true}],
 "problems": []}
```

Each process records itself at startup and refreshes `last_seen` as it serves requests (at most every 30 seconds), so the health checks a deployment already runs keep an idle one listed. `problems` holds `instances_not_shared` and `instances_versions_differ`, the same as the alerts of those names; both are also logged at startup.

## Run history

Every saved-query run is recorded: when, against which connection, by which key, through which front door (`transport`: `rest` or `mcp`), with what status, row count and duration. So is every ad-hoc run. How that history is written and kept is configurable:

| VARIABLE | DEFAULT | EFFECT |
|---|---|---|
| `QUERYAPIGATE_HISTORY_LIMIT` | `50` | Runs kept per saved-query version, unless a retention period is set. |
| `QUERYAPIGATE_HISTORY_RETENTION_DAYS` | unset | Keep **every** run for this many days instead of a per-version count; a sweep every 10 minutes deletes older ones. Best with a PostgreSQL metadata store. |
| `QUERYAPIGATE_HISTORY_SAMPLE_RATE` | `1` | Fraction of *successful* runs recorded, e.g. `0.1` for one in ten on a very busy query. Failed runs are always recorded. A run sampled out is still a live event. |
| `QUERYAPIGATE_HISTORY_FLUSH_INTERVAL` | `1` | Seconds between batched history writes. `0` writes each run inside its own request. |
| `QUERYAPIGATE_HISTORY_ADHOC_LIMIT` | `1000` | Ad-hoc runs kept in all, newest first, unless a retention period is set. |
| `QUERYAPIGATE_HISTORY_ADHOC_SQL` | `text` | What an ad-hoc run's entry keeps of its SQL: `text` (the first 4,000 characters, with `sql_truncated` when cut), `hash` (`sql_sha256` only - which statements repeat, never their text) or `none`. SQL can embed literal values; choose `hash` or `none` if those may be sensitive. |

**Batched writes.** A request never waits on its history entry: the run is queued in memory and a background thread writes everything queued in one transaction every `QUERYAPIGATE_HISTORY_FLUSH_INTERVAL` seconds. Reads in the same process always include its own queued runs (they are written first), so a history read right after a run shows it; another worker or instance sees it within one interval. A normal shutdown writes what is queued; a process killed outright loses at most one interval's worth. If the store can't keep up, at most 10,000 runs are queued per process and newer ones are dropped rather than slowing requests - counted in `/metrics` as `queryapigate_history_runs_total{outcome="dropped"}`, alongside `recorded`, `sampled_out` and `failed`, with `queryapigate_history_pending` for the current queue.

## Ad-hoc runs

`POST /execute_sql`, `POST /execute_mongo` (including streamed exports) and MCP's `execute_sql` tool are recorded too: the questions agents and people ask with no saved query behind them are the ones most worth being able to look back on. An ad-hoc entry has `"kind": "adhoc"`, `query` and `version` `null`, and:

- the caller (`key_name`), `transport`, `connection_name`, status, rows, duration, and for a failure its `error` and `code` - as for a saved run;
- the SQL, as `QUERYAPIGATE_HISTORY_ADHOC_SQL` says;
- `params`: the names of its bound parameters, **never their values**.

Ad-hoc runs are sampled like saved ones (`QUERYAPIGATE_HISTORY_SAMPLE_RATE`; failures always kept) and kept to `QUERYAPIGATE_HISTORY_ADHOC_LIMIT` in all, or the retention period if one is set. `queryapigate export` records its runs as saved-query runs by `key_name` `cli`.

```json
{"kind": "adhoc", "query": null, "version": null, "executed_at": "2026-10-04 09:12:40", "status":
"success",
 "rows": 12, "duration_ms": 9, "key_name": "agent", "transport": "mcp", "connection_name": "warehouse",
 "sql": "SELECT region, SUM(total) FROM orders WHERE placed_at ⩾ :since GROUP BY region", "params":
["since"],
 "request_id": "4be1f07c9d22"}
```

### `GET /api/v1/history`

Pages through every stored run, saved and ad-hoc, newest first - across a retention period, for example. Admin only (it shows every key's activity). `GET /api/v1/queries/{name}/history` is the same for one query.

| PARAMETER | MEANING |
|---|---|
| `kind` | `saved` or `adhoc`. Each item says which it is. |
| `query`, `version` | Only this saved query (and version). |
| `status` | `success` or `error`. |
| `key` | Only runs made with this API key (`admin` for the admin key). |
| `since`, `until` | `YYYY-MM-DD` or `YYYY-MM-DD HH:MM:SS`, in the server's local time; `since` inclusive, `until` exclusive. |
| `limit` | Runs per page, 1-1000 (default 100). |
| `cursor` | The previous page's `next_cursor`. |

```shell
curl -H 'X-API-Key: admin-key' 'localhost:5000/api/v1/history?key=partner&status=error&since=2026-10-01'
```

```json
{"items": [{"query": "film_by_id", "version": 3, "executed_at": "2026-10-01 14:02:11", "status":
"error",
             "error": "An error occurred while executing the SQL query", "key_name": "partner",
             "connection_name": "pg", "request_id": "9f2c41d07a1b"}],
 "next_cursor": "WyIyMDI2LTEwLTAxIDE0OjAyOjExIiwgNDgxMl0"}
```

`next_cursor` is `null` on the last page.

## Audit log

`GET /api/v1/audit` (admin only) is a durable record of administrative changes - distinct from Observability above, which covers live request/query traffic, not configuration changes. Every create, update or delete of an API key, role, connection or saved query appends one entry, newest first (moving a query between collections is `move_query`, listing the keys that gained or lost access; renaming one is `rename_collection`):

```json
{
  "items": [
    {"timestamp": "2026-09-24 10:03:11", "actor": "alice", "via": "token", "action": "update_key",
     "target": "acme-corp", "changes": {"allow_writes": {"from": false, "to": true}}},
    {"timestamp": "2026-09-24 10:01:47", "actor": "admin", "via": "break-glass", "action":
"create_connection",
     "target": "reporting",
     "changes": {"db": "postgres", "host": "db.internal", "password": "********", "active": true}}
  ],
  "total": 2, "actions": ["create_connection", "update_key"], "retention": 500
}
```

`actor` is the administrator's name (`admin` for the shared key); `via` says how they authenticated - `token` (their own admin token), `break-glass` (`QUERYAPIGATE_API_KEY`), `open` (no authentication configured), `cli` (a command on the server) or `startup`. Entries written before 0.16 have no `via`. Administrators and their tokens are audited too: `create_admin`, `update_admin`, `delete_admin`, `issue_admin_token`, `revoke_admin_token` - never a token's secret.

An update's `changes` is a diff of only the fields that actually changed (`{"field": {"from": ..., "to": ...}}`); a create or delete records a full snapshot of the entry instead, since there's no prior or remaining state to diff against. A connection's `password` is never included as a value in either form - masked as `********` in a snapshot (the same mask `GET /api/v1/connections/{name}` already uses) and reported only as the literal string `"changed"` in a diff, so the audit log itself never becomes a second place a real password leaks from. An API key's entry never includes its secret or hash, the same fields `GET /api/v1/api-keys` already omits. Capped at 500 most recent entries by default; older ones roll off, the same way a saved query's `execution_history` is capped per version. Set `QUERYAPIGATE_AUDIT_LOG_LIMIT` to raise or lower that cap for a busier server or a longer compliance-driven retention window - validated at startup, so a malformed value fails loudly rather than silently keeping the default. Unlike the streaming row cap, this one is always a positive count: each audit event trims the table back down to the cap in the same transaction as its own insert, so letting it grow without bound would mean an ever-larger table and index, not just more disk.

For retention a cap can never satisfy - keeping every entry indefinitely rather than a rolling window of however many - set `QUERYAPIGATE_AUDIT_LOG_EXPORT_FILE` to a path; every entry is also appended there, one JSON object per line, and that file is never capped or rewritten. The two writes are independent, so a problem with one (the export path's directory missing, say) never blocks the other.

### A tamper-evident chain

Since 0.25 (ADR 0012) each entry is a link in a hash chain: `seq` (1, 2, 3 … across every instance, with no gaps), the previous entry's `hash` as its `prev_hash`, and its own `hash` over both and the entry. Change, remove, insert or reorder an entry, and every hash from there on stops matching.

- **Keyed, or not.** With `QUERYAPIGATE_AUDIT_KEY` set (32 characters or more, kept outside the store like `QUERYAPIGATE_SECRET_KEY`), each `hash` is `hmac-sha256:...`: someone who can write to the store but hasn't the key can't re-sign an edit. Without it, `sha256:...` - consistent, but anyone with the store can recompute every hash after an edit; Settings says which is in effect. Setting a key is recorded (`audit_key_set`), and from then on an unkeyed entry - or a chain re-written unkeyed from the start - is a break.
- **Rotating the key:** `QUERYAPIGATE_AUDIT_KEY=<new>,<old>` - the first signs, every one verifies; startup records `audit_key_rotated`. Keep the old key listed for as long as entries signed with it are kept.

- **The rolling cap still applies:** the chain's anchor remembers the newest entry trimmed, so the oldest kept must follow it - removing entries from the start isn't mistaken for a trim.
- **Checking it:** `queryapigate audit verify` (exit 0 intact, 1 broken) or `GET /api/v1/audit/verify`, which name the first entry that doesn't verify; `queryapigate audit verify --file` checks a copy on its own - an export file, or what your SIEM received, as JSON lines or as the syslog lines a receiver writes - whose entries carry `seq`, `prev_hash` and `hash` too. Delivery to a SIEM is at least once, so its copy may hold an entry twice, or out of order after a retry: an exact repeat (the same `seq` and `hash`) is dropped and counted, and entries are checked in `seq` order; a repeat that differs is `conflicting_copies` - one of the two isn't what was written. Each server re-checks every 5 minutes: `audit_chain_broken` (critical).
- **What it can't show on its own:** the newest entries removed - the chain's end can't know what came after it - or a rewrite by someone holding both the store and the key. A copy outside the store, in your SIEM, is what shows those.
- **Upgrading to 0.25** chains the entries already stored, as they are, and records `audit_chain_started`: the chain proves nothing about them before that moment.

### Streaming the audit log to a SIEM

> **Stable** since 0.32, under the compatibility promise (what that means), tested on every push against rsyslog, syslog-ng and the OpenTelemetry Collector (ADR 0019). Outputs, structured-data parameters and record attributes may still be added; the message's MSGID, `[meta sequenceId]` and JSON body, and the attributes below, stay as they are. Limits: syslog over TCP has no acknowledgement - a receiver that dies without closing the connection can lose the batch sent into it (OTLP's is acknowledged); delivery is at least once; no UDP.

Every entry, as it's written, to your SIEM - Splunk, Microsoft Sentinel, Elastic, QRadar, or a log forwarder:

- **Syslog:** `QUERYAPIGATE_AUDIT_SYSLOG_URL=tls://siem.example.com:6514` (TLS, RFC 5425) or `tcp://host:514`; UDP is refused - it loses entries silently. RFC 5424 messages, octet-counted: facility 13 (log audit), the `action` as MSGID, `[meta sequenceId="<seq>"]`, and the entry as JSON with `seq`, `prev_hash` and `hash`. The server's certificate is checked against the system's CAs, or `QUERYAPIGATE_AUDIT_SYSLOG_CA`; `QUERYAPIGATE_AUDIT_SYSLOG_CERT` and `_KEY` give a client certificate, for mutual TLS. On Python 3.13 and later the check is strict (RFC 5280): a private CA's certificates need their Authority and Subject Key Identifiers - which `openssl` and every real CA add, but a hand-rolled script may not (*certificate verify failed: Missing Authority Key Identifier*).
- **OTLP logs:** `QUERYAPIGATE_AUDIT_OTLP=1` - OTLP/HTTP with JSON encoding to `OTEL_EXPORTER_OTLP_LOGS_ENDPOINT` (or `OTEL_EXPORTER_OTLP_ENDPOINT` + `/v1/logs`), with `OTEL_EXPORTER_OTLP[_LOGS]_HEADERS`. No extra package. Each entry a log record: the entry as its body, `queryapigate.audit.seq`, `.hash`, `.prev_hash`, `.action`, `.actor`, `.target` as attributes; `service.name` `queryapigate`, scope `queryapigate.audit`.

**Delivery survives the SIEM being down.** Each output's cursor - the last `seq` delivered - is in the store, and one instance delivers at a time (whichever holds the output's lease; another takes over if it stops). While an output is behind, the entries it hasn't had are kept past `QUERYAPIGATE_AUDIT_LOG_LIMIT` - up to ten times it. A new output starts with every entry the store holds. **At least once:** a batch that failed partway is sent again, so the SIEM may see an entry twice - drop the repeat by `seq` and `hash`. A batch counts as delivered once the OTLP endpoint answered 2xx, or syslog's connection took it; syslog over TCP has no acknowledgement, so a SIEM that dies without closing the connection can lose the batch sent into it (one that closes it, or refuses the client certificate, is noticed before anything is written). Where that matters, use OTLP, or a log forwarder next to QueryAPIGate.

`GET /api/v1/audit/delivery` shows each output: `delivered_seq`, `head_seq`, `behind`, `oldest_undelivered_at`, `last_delivered_at`, `last_error` and the `instance` delivering it. The `audit_delivery_behind` alert says when an output is more than five minutes behind (critical while it's also failing).

The admin UI's Audit Log tab filters this client-side over what it already fetched - an action dropdown (populated from whatever actions actually appear) and a search box matching actor, target or timestamp - and renders a snapshot's `changes` (a create/delete) with any unset field (`null`, `""`, `[]`) omitted rather than always listing every field, so a key or connection with few grants set doesn't read as a long, mostly empty list. A diff (an update) is unaffected by this - it never had unset fields in it to begin with, only whatever actually changed.

## Rate limiting and CORS

Both are off unless the server enables them (`QUERYAPIGATE_RATE_LIMIT`, `QUERYAPIGATE_CORS_ORIGINS`).

- **Rate limit.** When enabled, every response carries `X-RateLimit-Limit` (the quota) and `X-RateLimit-Remaining`. A client over its limit receives **429** with a `Retry-After` header (seconds) and `{"error": "Rate limit exceeded", "code": "rate_limited", "retry_after": 12, "request_id": "..."}`. `/health` is never limited.
- **Per-key rate limit.** An API key can also carry its own `rate_limit` (same grammar, e.g. `"100/minute"` - see Per-key rate limiting), checked *in addition to* the server-wide limit above, never instead of it - a response carries both header pairs when both apply (`X-RateLimit-Limit` / `X-RateLimit-Remaining` for the server-wide one, `X-RateLimit-Key-Limit` / `X-RateLimit-Key-Remaining` for the key's own), and a rejection from the key's own limit reads `{"error": "Rate limit exceeded for this API key", ...}` - distinguishable from the server-wide rejection's plain `"Rate limit exceeded"`.
- **Shared across instances.** With `QUERYAPIGATE_REDIS_URL` set, both limits are counted in Redis - one budget per client address and per key across every instance and process. If Redis fails, each process counts on its own until it's reachable again (`queryapigate_rate_limit_fallbacks_total`, and the `rate_limits_not_shared` alert).
- **CORS.** For listed origins the server answers preflight (`OPTIONS`) requests and adds `Access-Control-Allow-Origin` to responses, exposing `X-Page`, `X-Page-Size`, `X-Has-More`, `X-RateLimit-*` (server-wide and per-key), `Retry-After`, `X-Request-Id`, `X-Cache`, `ETag`, `Deprecation` and `Link` to the page's JavaScript. Allowed methods are `GET, POST, PATCH, DELETE, OPTIONS`; allowed request headers are `Content-Type`, `X-API-Key`, `Authorization` and `X-Request-Id`. Credentials (cookies) are not used.

## Admin UI

The admin UI - the QueryAPIGate Console - is at `/console` (ADR 0001). It covers the whole workflow: connections, the API Designer (a SQL editor with completion that knows your tables and columns, a schema browser, Explain, parameterize, run and save), the API Repository (versions, drafts and publishing, history, curl and CLI snippets, cache, metrics, who can reach each query), API keys and roles, the Access map, caching, metrics, the audit log, settings and help.

It is a client of the Management API and the runtime routes only - it adds no server-side logic of its own. Loading the page needs no API key; the requests it makes are gated exactly like any other client's, so a scoped (non-admin) key sees "only the admin key" messages wherever it reaches past its grants. It shares its API-key storage with `/docs` (the same browser-tab-only `sessionStorage` entry), so entering the key on one page covers both. Interface preferences (theme, table density, default result format) are kept in the browser.

The Console is built with React and TypeScript and ships prebuilt in the wheel and the Docker image - running it needs no Node.js. `/ui`, where an earlier, hand-written admin page used to be, redirects here.

# Interactive documentation

`/docs` (Swagger UI, backed by `/openapi.json`) documents the generic API and also lists **every saved query as its own endpoint**, generated from its latest version: its parameters with types, defaults, ranges and descriptions, and whether a connection must be named. The SQL text is never included.

The generic part is public. When the server sets `QUERYAPIGATE_API_KEY`, the saved-query part is only included for requests that carry the key - paste it into the box at the top of `/docs` (kept in that browser tab only) or send `X-API-Key` to `/openapi.json`.

Since `/openapi.json` is a standard OpenAPI 3.0 document, Postman and Insomnia can both import it directly by URL (Postman: *Import → Link*) to get a ready-made collection of every endpoint, including saved queries once you've supplied a key - no separate export step.

## The API catalogue

`GET /catalog` answers a different question than `/openapi.json`: not just *how* to call a saved query (parameters, types, connection) but *under what terms* - whether its response can be cached, whether the calling key specifically can write through it, and what rate limit governs the calling key itself. That information already exists (`cache_ttl` on the saved query, a key's own `rate_limit`, per-query write curation - see Per-query write curation) but was otherwise only visible on admin-only screens a scoped key can never reach:

```json
{
  "queries": [
    {"name": "top_rented_films", "version": 1, "description": "Films ranked by number of rentals",
     "tags": ["reporting", "films"], "collection": "reporting", "connection_name": "rental_db",
     "parameters": {}, "cache_ttl": 60, "can_write": false}
  ],
  "caller": {
    "name": "acme-corp", "admin": false, "allow_writes": false, "allowed_write_ops": null,
    "allowed_tables": null, "rate_limit": "200/hour", "server_rate_limit": null
  }
}
```

`queries` is scoped exactly like `/openapi.json`'s saved-query list - a query this caller cannot reach through `/q/<name>` is never listed here either, so the catalogue never shows a caller something it can't actually use. `cache_ttl` is `null` when the query isn't cached; `can_write` reflects this specific caller (a query with no per-query write curation for them still reads `false` even if their key has blanket `allow_writes`, since blanket access is already visible on their own key). `caller.rate_limit` is this key's own additional limit (`null` if it has none of its own); `caller.server_rate_limit` is `QUERYAPIGATE_RATE_LIMIT`, checked in addition to it, never instead of it. Unlike `/openapi.json`, `/catalog` is never public - it requires authentication like any other functional endpoint, since the whole point is answering "what can /use," which needs a resolved caller to mean anything.

# Client SDKs

`/openapi.json` is a complete, valid OpenAPI 3.0 document (checked in CI against the official validator), so a typed client for Java, TypeScript, Go, or any of the ~50 languages `openapi-generator` supports costs nothing in application code - generate it from the running server's own spec:

```shell
npx @openapitools/openapi-generator-cli generate \
  -i http://127.0.0.1:5000/openapi.json -g java -o clients/java
# or: -g typescript-fetch, -g go, -g python, ...
```

This is deliberately not something QueryAPIGate ships pre-generated: the spec already includes every saved query as its own typed `/q/<name>` endpoint (see Run a saved query), so a client generated against *your* server reflects *your* saved queries, not a generic snapshot.

## Errors

Every error - from `/api/v1`, the runtime routes (`/q/<name>`, `/execute_sql`, `/catalog`, …), `queryapigate events` and MCP tool calls - has the same shape:

```json
{"error": "This API key may only query these tables: public.film. ...", "code": "table_not_allowed",
 "request_id": "9f2c41d07a1b"}
```

- **`code`** is stable: branch on it. It is part of the compatibility promise - an existing code is never renamed or given a different meaning, though new ones may be added (treat an unknown code by its HTTP status).
- **`error`** is for people. Its wording may change in any release.
- **`request_id`** is also the response's `X-Request-Id` and appears on the server's log lines for the request, so a support conversation can find them. (`queryapigate events` errors have no `request_id`.)
- Some errors add fields: `errors` (a map of parameter name to problem, for `param_invalid`, `param_required` and invalid query definitions), `retry_after` (seconds, for `rate_limited`, also sent as `Retry-After`), `timeout` (for `query_timeout`) and `detail`.
- **`detail`** carries the database's own message when a query fails - except for a saved query (`/q/<name>`) called with a scoped key, which gets only the generic error, since the database's text can reveal schema details to a caller who didn't write the SQL. The full message is always in the server log.
- **MCP:** a tool error is `isError: true` with the message as text, as before, and `{"error", "code"}` (plus `retry_after` when rate limited) in `structuredContent`.

An error with no more specific code gets the one for its status: `invalid_request` (400), `unauthorized` (401), `forbidden` (403), `not_found` (404), `method_not_allowed` (405), `conflict` (409), `precondition_failed` (412), `payload_too_large` (413), `unsupported_media_type` (415), `rate_limited` (429), `internal_error` (500), `upstream_failed` (502), `unavailable` (503), `query_timeout` (504).

| CODE | STATUS | MEANING |
|---|---|---|
| `invalid_body` | 400 | The request body isn't valid JSON, isn't an object, or has a field with a wrong value |
| `unknown_field` | 400 | The body has a field this route doesn't take |
| `invalid_name` | 400 | A query, key, role, collection or connection name with characters it can't have |
| `invalid_paging` | 400 | `page` / `page_size` not positive, or `page_size` over `QUERYAPIGATE_MAX_PAGE_SIZE` |
| `invalid_format` | 400 | An unsupported `format` |
| `invalid_timeout` | 400 | `timeout` isn't a positive number of seconds |
| `invalid_stream` | 400 | `stream=true` with a format it can't stream, or with `page` / `page_size` |
| `invalid_filter` | 400 | A bad history/audit filter ( `status` , `kind` , `since` , `until` , `limit` , `cursor` ) or Mongo filter |
| `invalid_bundle` | 400 | A collection bundle that isn't one, or is malformed |
| `invalid_config` | 400 | A configuration file (ADR 0014) that isn't one, or has problems - each one listed in `problems` |
| `redis_unavailable` | 503 | Forgetting a caller needed Redis - which holds their usage counts - and it couldn't be reached; the store's were deleted, run it again |
| `secret_unavailable` | 503 | A secret provider couldn't give a connection's secret, and no value still within its time was held (ADR 0015) |
| `embedding_unavailable` | 503 | A query has an `embed` parameter but semantic search isn't configured ( `QUERYAPIGATE_EMBEDDING_URL` / `_MODEL` ) |
| `embedding_failed` | 502 | The embeddings endpoint couldn't be reached or answered in a shape the server couldn't read |
| `config_managed` | 409 | With `QUERYAPIGATE_CONFIG_LOCKED` , a change to something a configuration file manages - change it in the file |
| `sql_required` | 400 | No SQL given |
| `connection_required` | 400 | No connection named, and the saved query has none |
| `collection_required` | 400 | No collection named |
| `table_required` | 400 | `/table_ddl` without `table` |
| `multiple_statements` | 400 | More than one SQL statement |
| `param_required` | 400 | A required parameter (or placeholder) has no value; `errors` names each |
| `param_invalid` | 400 | A parameter's value breaks its rules; `errors` says how |
| `param_from_claim` | 400 | The call sent a parameter the query takes from the caller's sign-in token |
| `reason_required` | 400 | Deleting a connection without a `reason` |
| `unsupported_database` | 400 | A database type QueryAPIGate doesn't support |

| CODE | STATUS | MEANING |
|---|---|---|
| `unsupported_operation` | 400 | The operation isn't available for this database type (listing databases, schema, DDL) |
| `wrong_connection_type` | 400 | A Mongo call on a SQL connection |
| `invalid_watermark` | 400 | An export's incremental column isn't in the result, or holds values that can't be compared |
| `unauthorized` | 401 | Missing or wrong `X-API-Key` or bearer token |
| `insufficient_scope` | 403 | On the MCP server: a valid access token that doesn't carry `QUERYAPIGATE_MCP_SCOPE` |
| `admin_only` | 403 | Only an administrator may do this (manage the server, query or browse another database) |
| `open_access_remote` | 403 | Open-access mode (no API key, administrator, JWT or MCP OAuth configured) answers only local clients; set `QUERYAPIGATE_API_KEY` to allow other hosts |
| `role_forbidden` | 403 | The administrator's role can't do this; `capability` names what it needed, `role` the caller's role. For a move or merge that would widen access, `gaining` lists the keys and roles |
| `workspace_required` | 403 | An administrator with no role in the workspace the call is for - only in others, which `workspaces` lists (Workspaces) |
| `connection_forbidden` | 403 | This key may not use this connection |
| `store_not_allowed` | 403 | A connection was pointed at the server's own metadata store (`queryapigate.db`) |
| `connection_inactive` | 403 | The connection is switched off |
| `read_only` | 403 | A write statement where writes aren't allowed (always, over MCP) |
| `write_op_not_allowed` | 403 | A write this key's `allowed_write_ops` doesn't include |
| `table_not_allowed` | 403 | A table outside this key's `allowed_tables`, or a table function (`read_parquet(...)`) for such a key |
| `exports_unavailable` | 500 | Exports and destination tests need DuckDB (`pip install "queryapigate[duckdb]"`), which isn't installed |
| `format_unavailable` | 500 | `format=parquet` on a server without DuckDB (`pip install "queryapigate[duckdb]"`) |
| `path_not_allowed` | 403 | A DuckDB connection was asked to read a file or URL outside its `allowed_paths` |
| `table_check_unsupported` | 403 | `allowed_tables` can't be enforced on this database type, so the query is refused |
| `table_check_failed` | 403 | The SQL couldn't be analysed to enforce `allowed_tables`, so it is refused |
| `mask_unresolvable` | 403 | The caller has column-masking rules, and a result column couldn't be traced to its source columns - so the query is refused rather than served unmasked; the message says which and why |
| `mongo_operator_forbidden` | 403 | A Mongo operator that runs server-side JavaScript |

| CODE | STATUS | MEANING |
| --- | --- | --- |
| `sign_in_required` | 403 | The query takes values from a sign-in token; call it with one |
| `claim_missing` | 403 | The sign-in token lacks a claim the query needs |
| `claim_binding_required` | 403 | The caller's role has `require_claim_binding`, and the query takes no parameter from the sign-in token |
| `forbidden` | 403 | Any other refusal |
| `query_not_found` | 404 | No such saved query |
| `mcp_disabled` | 404 | The saved query exists but isn't exposed as an MCP tool (its `mcp` is off); it still serves over REST |
| `version_not_found` | 404 | No such version - or a draft, to a key that can't see drafts |
| `not_published` | 404 | The query has no published version |
| `connection_not_found` | 404 | No such connection |
| `key_not_found`, `role_not_found` | 404 | No such API key or role |
| `key_rotation_pending` | 409 | The key's old secret from its last rotation still works (`previous_expires_at`) - end it (`DELETE /api/v1/api-keys/{name}/previous`) or wait, then rotate again |
| `key_secret_taken` | 409 | A configuration file gives a key the `secret_sha256` another key already has - each key needs its own secret |
| `previous_secret_not_found` | 404 | The key has no old secret that still works - nothing to end |
| `admin_not_found`, `token_not_found` | 404 | No such administrator, or no such token of theirs |
| `workspace_not_found`, `member_not_found` | 404 | No such workspace - or none the caller may see - or that administrator isn't a member of it |
| `destination_not_found`, `export_not_found` | 404 | No such destination or export |
| `collection_not_found` | 404 | No such collection (or it is empty) |
| `cache_entry_not_found` | 404 | No such cached response |
| `table_not_found` | 404 | No such table on the connection |
| `database_file_not_found` | 404 | A SQLite/DuckDB connection's file doesn't exist |
| `query_exists`, `connection_exists`, `key_exists`, `role_exists`, `collection_exists` | 409 | The name is taken |
| `admin_exists` | 409 | An administrator by that name exists |
| `workspace_exists` | 409 | A workspace by that name exists |

| CODE | STATUS | MEANING |
| --- | --- | --- |
| `workspace_not_empty`, `workspace_protected` | 409 | Removing a workspace that still owns something ( `contents` counts it), or `default` |
| `destination_exists`, `export_exists` | 409 | A destination or export by that name exists |
| `export_running` | 409 | Another run of the export holds its lease; retry after `retry_after` seconds |
| `destination_in_use` | 409 | An export still writes to the destination |
| `name_taken` | 409 | An administrator and an API key can't share a name |
| `sso_managed` | 409 | Changing the role of an administrator who signs in through SSO - their groups decide it |
| `sso_only` | 409 | A personal token for an administrator who signs in through SSO, with `QUERYAPIGATE_SSO_ONLY` on |
| `email_required` | 409 | Marking an administrator for SSO before they have an email to match |
| `sso_ambiguous` | 409 | Two administrators wait for an SSO sign-in with the same email |
| `write_request_not_found` | 404 | No agent write request by that id - or not this caller's, for this tool |
| `approval_pending` | 409 | An agent's write isn't approved yet |
| `approval_rejected` | 403 | A person rejected the agent's write; the message gives their reason |
| `approval_expired` | 409 | The agent's write wasn't approved, or run once approved, in time - ask again |
| `approval_mismatch` | 403 | The arguments aren't the ones a person approved - an approval is for exactly one write |
| `write_request_closed` | 409 | Approving or rejecting a write request that isn't pending |
| `write_running` | 409 | That approved write is running now |
| `own_write` | 403 | Approving or rejecting a write asked for under your own name - another approver must |
| `too_many_pending_writes` | 429 | An agent has 100 writes waiting for approval already |
| `idempotency_conflict` | 409 | An insert's idempotency key was used before with other arguments |
| `too_many_rows_affected` | 409 | An agent's write would have changed more rows than its query's `max_affected_rows` - rolled back, nothing changed |
| `last_owner` | 409 | The change would leave no active owner while `QUERYAPIGATE_API_KEY` isn't set |
| `examples_conflict` | 409 | Loading the examples would overwrite things that aren't examples |
| `precondition_failed` | 412 | `If-Match` names a version of the resource that is no longer current |
| `rate_limited` | 429 | Over `QUERYAPIGATE_RATE_LIMIT` or the key's own `rate_limit`; retry after `retry_after` seconds |

| CODE | STATUS | MEANING |
|---|---|---|
| `quota_exceeded` | 429 | The caller's usage quota for this period is used up; `quota` says which (`calls`, `rows`), `resets_at` when it resets |
| `too_many_concurrent` | 429 | The caller already has `max_concurrent` queries running; retry when one finishes (`Retry-After: 1`) |
| `query_failed` | 500 | The database rejected the query (`detail` has why, where shown) |
| `driver_missing` | 500 | The database driver isn't installed on the server, or is too old (DuckDB before 1.2) |
| `connection_misconfigured` | 400, 500 | The connection's settings are incomplete (no database file, a missing environment variable, JDBC fields) |
| `secret_key_required`, `password_undecryptable` | 500, 409 | An encrypted connection password can't be decrypted: `QUERYAPIGATE_SECRET_KEY` is unset, or lists no key it was encrypted with (500). From `POST` `/api/v1/settings/rekey`: no key is set, or a stored password no listed key decrypts - nothing changed, `values` names them (409) |
| `table_check_unavailable` | 500 | `allowed_tables` needs the `sqlglot` package, which isn't installed |
| `masking_unavailable` | 500 | Column masking needs the `sqlglot` package (`queryapigate[flow]`), which isn't installed |
| `internal_error` | 500 | Anything unexpected; the log has the traceback under `request_id` |
| `connection_failed` | 502 | The database couldn't be reached |
| `destination_unreachable` | 502 | A destination couldn't be written to - its url, credentials or permissions |
| `too_many_streams` | 503 | Every live-event stream slot is taken; retry shortly |
| `query_timeout` | 504 | The query ran past its time limit and was cancelled |